MATH SURVEY

2004 - 2005 Volume LXXIX

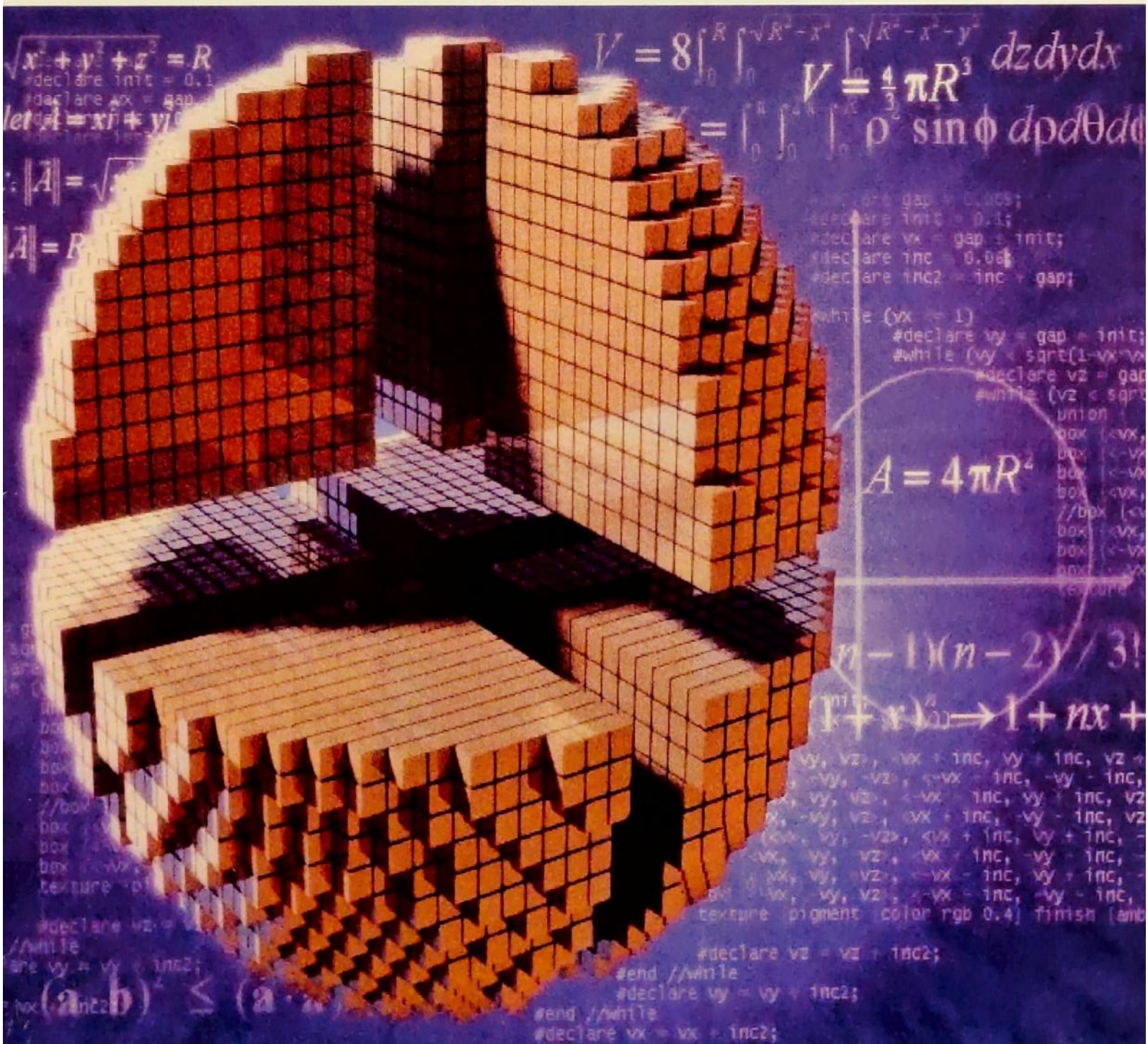


Table of Contents

Cover photo credit: www.brinkster.com/mulliganan

Editor's Note

Dear Readers,

Welcome to Math Survey! This magazine is proud to again address many topics of interest from golden ratio to surreal numbers. This set of articles is contributed by a truly talented student body interested in mathematics. Returning from last year, a good mix of brainteasers and math jokes has again been selected for your pleasure.

It is my most sincere hope that you will enjoy Math Survey. It truly would vindicate our hard work in putting together this magazine, making us sleep easier at night. Only through the continual effort and dedication of all the editors and staff was this publication possible. I would like to express my deepest gratitude to everyone who contributed to this issue.

On behalf of the magazine, I would also like to thank the Parents Association for its generous support; the Alumni Association for its contributions, and everyone who is holding a copy of the magazine now. If not for your loyal readership, Math Survey would not be celebrating its seventy-eighth volume.

Our appreciation also goes to Ms. Avigdor and other Honors math teachers, whose encouragement of their classes to submit Math Fair papers have also inspired them to contribute their delightful articles to us. In addition, we would like to thank Mr. Stern and Mr. Granovskiy for their support and encouragement in making this issue one of the first to use some LaTeX. And last but not least, we would like to thank Mr. Jaye whose support and encouragement in the toughest times helped pull us through.

For many of us, this is our last year working on the magazine. I will certainly miss being part of Math Survey and all the wonderful staff. Math Survey will certainly continue to prosper under capable leadership and broaden its horizons. I must urge all of you to contribute again next year if you could as after all, our contributors are the heart of the spirit of Math Survey. Meanwhile, I would urge all of you to enjoy this issue. May it open your mind to new and better things.

Sincerely,

Yuetian (Peak) Xu
Editor-in-Chief

Feature Paper Note

One salient characteristic of Stuyvesant students is their eagerness to participate in various mathematics and science competitions. This attribute is validated by the remarkable performance of Stuyvesant High School in many math/science competitions.

Many competitions require a research paper that demonstrates the students' originality and depth of comprehension on their specific research discipline. Some of these competitions are the Siemens Westinghouse, Intel, NYCSEF, and ISEF.

However, writing a research paper could be a daunting process filled with questions on the format and content of a "successful competitive paper."

Therefore, as an exclusive feature, the *Math Survey* has decided to include a research paper written by Yuetian (Peak) Xu who was selected as the semifinalist in the Siemens Westinghouse and the Intel competition. His paper also qualified him as a finalist for the ISEF. Yuetian Xu's paper is "A Practical Implementation of Gradient-based Convolutional Networks in Handwriting Recognition." This paper can be found on page 45.

I hope that this paper would provide you with a general concept of a well-written research paper. Perhaps, the most important ingredients to a successful research paper are your passion for your research topic, your creativity, integrity, and effort.

Many thanks to Peak for sharing his paper. Above all, special thanks and appreciation to the teachers, administrators and mentors who supported us in our many educational endeavors.

Best regards for success!

Steven Mon
Managing Editor

Vedic Mathematics

Ganesh Ramnarine

In a school with so many advanced math courses, it's quite easy to forget about the basics. Here, I present a system to help with mental arithmetic: Vedic mathematics. Vedic mathematics originates in the ancient Indian teachings known as the Vedas, which also is the foundation of Hinduism. In addition to helping with basic arithmetic, it can be used in higher mathematics such as calculus and linear algebra. It is based on sixteen "sutras" and a number of sub-sutras, or corollaries.

One sutra is निखिलं नवतश्चरमं दशतः, which translates to "All from 9 and the last from 10." This is the simplest one, which allows you to subtract numbers from any multiple of 10. The sutra means: subtract all the first numbers from 9, and the last from 10 to get your answer. For example: the number 1000 – 283 can have its digits expressed as (9 – 2), (9 – 8) and (10 – 3), which gives us: 717. If we wanted to calculate something such as 2000 – 29, we could consider the number 1000 – 29 and add 1000 to it. So the digits would be given by (9 – 0), (9 – 2) and (10 – 9), which gives us 971. If we add 1000 to the answer, we get 1971. This may seem overly simple, but I'm sure you have seen people make these kinds of trivial mistakes on an important math test.

A sub-sutra of this is यावदूनं तावदूनीकत्य वर्गं च योजयेत्, "Whatever the deficiency, lessen by that amount and set up the square of the deficiency." This is very useful for calculating squares of numbers near multiples of ten. An easy example is 9^2. The deficiency is from the nearest multiple of 10, so it would be 1. So $9^2 = (9-1)(10)+1^2 = (8)(10)+1 = 81$. For numbers larger than the nearest multiple of ten, we consider the surplus, rather than the deficiency. $14^2 = (14+4)(10)+4^2 = (18)(10)+16 = 196$. This makes squaring of large numbers near powers of ten into simple algebra.

$997^2 = (997-3)(1000)+3^2 = (994)(1000)+9 = 994009$. If we consider multiples that are not powers of 10, it becomes less simple, but still faster and easier than squaring by hand.

$206^2 = (206+6)(200)+6^2 = (212)(200)+36 = 42436$.

A very powerful sutra is ऊर्ध्वतिर्यग्भ्यामं, "Vertically and Crosswise." Using this, mental multiplication becomes very easy. For example, let's say we want to multiply 64 by 99. 64 is 36 less than 100 and 99 is 1 less than 100. We can picture the problem set up this way: $\begin{matrix} 64 & 36 \\ 99 & 1 \end{matrix}$ (this is where the first sutra comes in handy). We multiply the last row vertically to obtain the last two digits, (36 × 1) = 36. We subtract crosswise to get the first two digits, (99 – 36) = (64 – 1) = 63. Thus, (64 × 99) = 6336. The process is similar for higher numbers, say we wanted to multiply 102 by 121. We can then picture the problem as: $\begin{matrix} 102 & 2 \\ 121 & 21 \end{matrix}$. The last digits are given by (2 × 21) = 42. Since the numbers are

number less than 1. For example, let the ratio be 1/2. It follows that $y = 2x - 4$. Square this and subtract it from 16. What is left is the square of x. We have: $x^2 = 16x - 4x^2$. Cancel an x and you have one rational solution for x. Choose another ratio and you'll find another x. The beauty in this method is that the 16 cancels upon subtraction. This allows us to reduce the equation to a linear form. The variable x always ends up being rational.

3 Preliminaries for the Pythagorean formulas

By the fundamental theorem of arithmetic, every number has a unique prime factorization: $p^a q^b r^c \ldots$, where $p, q, r \ldots$ are primes and $a, b, c \ldots$ are nonnegative integers. A number is a perfect square if and only it each of the exponents is even. This is intuitively obvious. A square can be split into two equal factors. Each factor must have the same number of primes p, primes q, primes r etc. Thus, in total, there are an even number of primes p, primes q, primes r etc. Conversely, if a number has all such exponents even, we can split it into two factors with the number of each prime in each factor equal. The given number would then be expressed as the product of two equal numbers and would therefore be a perfect square.

3.1 The Pythagorean relationship: $a^2 + b^2 = c^2$

If integers a, b, c satisfy the Pythagorean relationship they are said to form a Pythagorean triple. If the numbers are pairwise coprime they are said to form a primitive Pythagorean triple. For example, 10, 24, and 26 form a Pythagorean triple but not a primitive one because they share a factor of 2. The numbers 3, 4, and 5 form a primitive triple because they are relatively prime in pairs. Let us find a formula to give us all possible primitive triples. Our search for an all encompassing formula will require a few steps. First, we will reduce our search a bit by showing that all Pythagorean triples are either primitive or are multiples of primitive triples. We will then show that $c+a$ and $c-a$, (when b is odd) are coprime except for a factor of two. After that we will show that half of each of those two numbers is also a square. We will then state this algebraically and arrive at our formula. As a check we will prove that a triple made by our formula will always be primitive.

We now show that if we can find one primitive triple we can generate infinitely more non-primitive triples by multiplying each number by a common factor. This is simple to prove. Let's say we have a primitive triple (a,b,c). We know that $a^2 + b^2 = c^2$, by definition. Multiply both sides of the equation by k^2 to get $(ak)^2 + (bk)^2 = (ck)^2$. Thus, (ak,bk,ck) is a Pythagorean triple. Here's an example: $(a,b,c) = (3,4,5)$, $k = 7$, $(ak,bk,ck) = (21,28,35)$, $21^2 + 28^2 = 35^2$.

We should also note that if any two numbers share a common factor, the third number will share that factor also. This is easy to see by simple algebraic manipulation. For example, let's say b and c share a common factor called k. Plug in mk for b and nk for c and isolate a^2. It easily follows that a is divisible by k. So we know that a primitive triple generates an infinite number of non-primitive triples and that being relatively prime as a group is not possible without being pairwise relatively prime. Now we can focus our attention on primitive triples.

Using modular arithmetic we quickly see that in a primitive triple one of the smaller numbers (a and b) must be even and the other odd. Furthermore, c must be odd. Without loss of generality let b be even. We can now show that $c+a$ and $c-a$ are coprime except for a factor of two. To prove this, add the two numbers to obtain $2c$ and subtract to obtain $2a$. If two numbers are divisible by the same number, so are their sum and difference. The GCD of $2a$ and $2c$ is denoted by $(2a,2b)$. It is easy to

prove using our discussion of factorization that this is equivalent to $2(a,b)$. However, as a and c are coprime, (a,b) is equal to one. Thus, $(2a,2b) = 2(a,b) = 2$. This means that if a number divides both the sum and the difference of a and c, it must be 2. Now, if we divide both the sum and the difference by 2 we will have two relatively prime numbers (because if two numbers are divided by their GCD the resulting numbers are coprime).

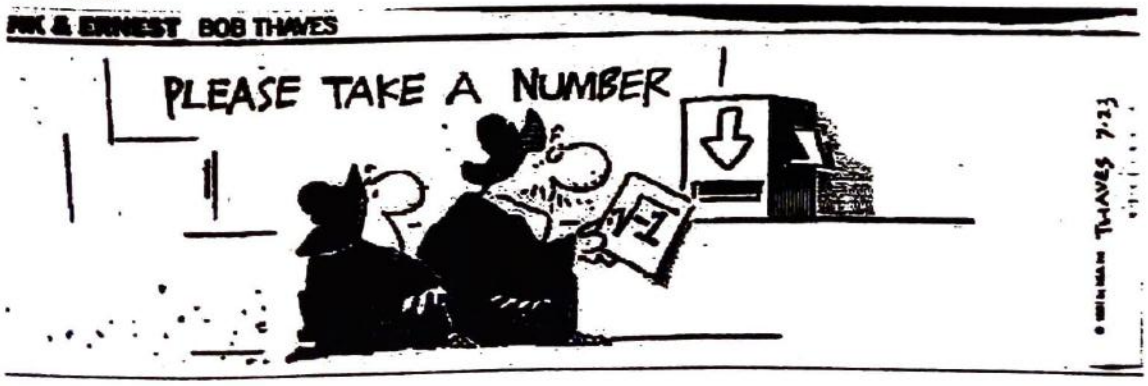
Half the sum and half the difference of a and c are perfect squares. This is because these numbers are relatively prime numbers that multiply to a perfect square ($(b/2)^2$). We have already shown that the sum and difference halved are relatively prime. We can show that they multiply to $(b/2)^2$ by rearranging the Pythagorean relationship: $b^2 = (c+a)(c-a)$, $b^2/4 = (b/2)^2 = (c+a)/2 \times (c-a)/2$. Using our discussion of the factorization of squares into primes, it is easy to see that if two coprime numbers multiply to produce a square, each will be a square.

We now state the above result in algebraic terms and derive our formula. We can say $(c+a)/2 = u^2$ and $(c-a)/2 = v^2$. Solving for c and a we can say that $c = u^2 + v^2$ and $a = u^2 - v^2$. However, for a and c to be odd only one of u and v must be odd. Also, u and v must be relatively prime or else a and c will not be relatively prime. If we plug these new forms for a and c into our Pythagorean relationship we see that $b = 2uv$. So we have our formula (or, actually, formulas): $b = 2uv$, $a = u^2 - v^2$, $c = u^2 + v^2$. Our formulas require that u and v be relatively prime. Can we find u and v for all primitive triples? Yes. To find u and v for a primitive triple just go through the same process we just went through but use the particular values you have in the derivation.

All that remains to be proven is that these formulas will always produce a primitive triple with the proper u and v. We can prove this by adding and subtracting b and c (and their representations in terms of u and v). If a prime p divides both the b and c, it will divide their sum and difference. Then, we come to the conclusion that p will divide the square root of the sum and the square root of the difference of u and v. If this is true, then p will divide the sum and difference of those square roots. This means that p will be two or will divide u and v. However, p cannot be two because b and c are not both even. Also, p cannot divide u and v because they were chosen to be relatively prime. We arrive at a contradiction and the triple is indeed primitive.

4 Conclusion

Diophantus was a brilliant mathematician who worked on many, many problems. His solutions were both beautiful and effective. His influence lives on in the field of Diophantine analysis. He will always be associated with the work of the number theoretical genius, Pierre Fermat, and his last theorem.



Pirates, Bribing, and Backward Induction

Mikhail Aronov

We have all been told, at one point or another, to think rationally. In everyday situations, this simply means making logical, objective decisions; in math however, rational thinking is used to solve mathematical problems. When there are many people, each having to make related decisions, entire systems are created. In mathematics, such systems are studied with game theory. Game theory uses simple games with simple rules in order to study the more complex systems. Game theory also employs different mathematical strategies. One example of a widely used strategy is backward induction. An easy way to show how this strategy works is through the pirate game.

The scenario for the game is first set – a certain number of pirates have gotten their hands on chest with 100 gold coins. The pirates have a hierarchy; each pirate has his own level of fierceness, indicated by a natural number: pirate 1 is the least fierce, pirate 2 is more fierce than pirate 1 but less fierce than pirate 3… and so on. The fiercest pirate in the group proposes a way to divide the gold. Everyone in the group votes, including the fiercest pirate himself. If 50% or more of the pirates vote "aye" on the proposal, it is passed and the gold given out accordingly. However, if the proposal is not passed, the pirate who made the proposal walks the plank, and the next pirate in level of fierceness gets to make the proposal. This continues until someone makes a proposal that gets passed.

First of all, each pirate wants as much gold as he can get without getting thrown overboard. Each pirate enjoys seeing others thrown overboard, but if given a choice, he will take money instead None of the pirates enjoy getting thrown overboard, and will do anything possible to avoid it. This implies that the pirate who is making the proposal can "bribe" other pirates into voting for him. However, even a bribed pirate will never vote for a proposal if another proposal will get him more money.

In example, we can start off with the situation involving just two pirates – 1 and 2, with 2 being the fiercest. Since there are only 2 pirates, pirate 2 can make the following proposal: "I get 100 coins." Pirate 1 will, of course, vote against, since it gets him no money, but pirate 2 will vote for himself, thus getting the 50% he needs for the proposal to be passed.

While this the solution is simple and is very obvious, it ultimately becomes the basis for all solutions to the game. When we go on to analyze the pirate game with 3 pirates, the game takes on an interesting twist. This time, pirate 3 makes the proposal. Since he is, like all the pirates, rational, his reasoning will be the following: "If I propose to give myself 100 coins, pirate 2 will vote "no" because he wants me to get thrown overboard, after which he will make the proposal and get 100 coins. Pirate 1 will vote "no" because he will not get any gold either way, but he wants me to get thrown overboard because he is a pirate." Here, we see how the strategy of backward induction is used for the rational reasoning of the pirates: the pirate looks at what would happen if the game was reduced by 1 pirate (in this case, if he got thrown overboard).

For pirate 3 to reach a solution in which he will get money and not die, he must bribe one of the pirates for the vote, which he needs in order for the proposal to get passed. He won't bribe pirate 2, since no matter how much gold he offers, pirate 2 will be able to receive the same amount or more if pirate 3 is thrown overboard. However, pirate 1 will not get any gold if pirate 3 is eliminated, so pirate

3 will bribe him. Because of this, pirate 3 will get pirate 1's vote no matter how much gold he gives him. Thus, he will propose that pirate 1 gets 1 coin, the lowest amount possible, and he himself get 99 coins. Pirate 1 will vote yes, as he prefers money to getting someone thrown overboard. Pirate 2 will vote "no," and pirate 3 will of course vote for himself. Thus, pirate 3 will receive 2/3, or about 67% of the vote, which is more than enough to have the proposal passed.

From the game with 3 pirates, we see cooperation between the players. Each pirate knows that, in order to stay alive, he must be able to cooperate with other pirates – ultimately by bribing them. Although, we hope this does not happen in reality, this is also a very good example of cooperative economics. This also leads to a conclusion where more than 1 player receives a profit, and all of the resources are used.

We continue with our analysis – this time with 4 pirates. Pirate 4 will make a proposal, but once again he cannot keep all the gold for himself, to avoid getting thrown overboard. He reasons: "If I bribe pirate 3, he will of course vote no, because if I get thrown overboard, there will be a 3 pirate game, and he will receive 99 coins. I can bribe pirate 1, but I will need to give him more than 1 coin, since he will get 1 coin in a 3 pirate game and wants to see me thrown overboard. However, if I bribe pirate 2 with only 1 coin, he will vote "yes," because if I get thrown overboard, he will receive no coins." Thus, pirate 4 will propose to give himself 99 coins and pirate 2 the remaining coin. With this proposal, pirate 4 will get the 50% of the vote, and the proposal will be passed. Therefore, in a game with 4 pirates, pirate 2 will get 1 coin, and pirate 4 will get 99.

From this we see that the pirate making the proposal must bribe those pirates that will receive nothing if he is thrown overboard. An odd-numbered pirate will bribe all other odd-numbered pirates, and an even-numbered pirate will bribe all even-numbered pirates. The game continues in this same fashion until we reach a point where the coins begin to "run out." This will first happen in a 199-pirate game. Pirate 199 will bribe all other odd numbers, that is, 1 – 197. However, this uses up 99 coins, and the pirate himself is also left with only 1 coin. The same goes for pirate 200 – he will bribe the even numbers from 2 – 198 and will be left with 1 coin.

Pirate 201 will not get any coins at all. He will have to bribe all the odd-numbered pirates preceding him, that is, 1 – 199. However, this uses up all of the 100 coins, so pirate 201 must give up everything in order to stay alive. Similarly, pirate 202 will bribe pirates 2 – 200 and will be left with no coins, and will also stay alive.

Pirate 203, however, is not so lucky. Even if he bribes the odd-numbered pirates, 1 – 199, pirates 200, 201, and 202 will vote "no," as they will not receive any money either way, and enjoy seeing others get thrown overboard. Thus, pirate 203 will get 102 votes against the proposal and 101 for the proposal. No matter what he does, pirate 203 is eliminated.

From here, it is very tempting to believe that all further pirates are eliminated as well. However, this is not the case, as the very next pirate, 204, will remain alive. This happens because the previous pirate, 203, knows that if pirate 204 is eliminated, he will share the same fate. Therefore, he will vote for pirate 204 not matter what. Pirate 204 must bribe other pirates, but he will no longer bribe the even-numbered pirates, even though he himself has an even number. This is because pirate 203 is also thrown overboard if pirate 204 is eliminated, thus distorting the sequence of pirate bribing. In other words, if pirate 204 is thrown overboard, the game eventually reduces to a 202-pirate game (instead of

203), and the odd-numbered pirates are those who do not get anything. Therefore, pirate 204 will bribe pirates 1 – 199, getting 100 votes, and will also get the vote of pirate 203, as we have already found out. Adding his own vote, he will get a total of 102 votes, which is enough to keep him from getting eliminated.

Following this, we can figure out that pirates 205, 206, and 207 are all eliminated. It is even more tempting after this to simply state that all further pirates will be eliminated as well. However, this is not the case when we reach pirate 208. Because he must bribe the pirates that will not receive any money if he were to be eliminated, pirate 208 will bribe the even-numbered pirates, since 204 would bribe the odd-numbered ones. In addition to those 100 votes, he will also get the votes of pirates 205, 206, and 207 – as they want to avoid getting thrown overboard. Adding his own vote, he will get 104 votes, which is enough to have the proposal passed.

By now, we have seen that, after pirate 200, pirates 201, 202, 204, and 208 were the only ones who survived. If we were to continue doing this, we would see that pirates 216, 232, 264, 328, and so on would be the only ones capable of staying alive. We can conclude that after the 200th pirate, only a pirate who's number is a power of 2 added to 200 will remain alive, or $200 + 2^n$. Moreover, we saw that the way of determining which pirates will be bribed is different with more than 200 pirates. We saw that pirates $201 = 200 + 2^0$ and $204 = 200 + 2^2$ bribed odd pirates, while pirates $202 = 200 + 2^1$ and $208 = 200 + 2^3$ the even ones. Thus, in a $200 + 2^n$-pirate game, the odd pirates are bribed if n is even and the even pirates are bribed in n is odd. To better visualize all results, I illustrated them with a graph, shown in Figure 1.

Fox Trot

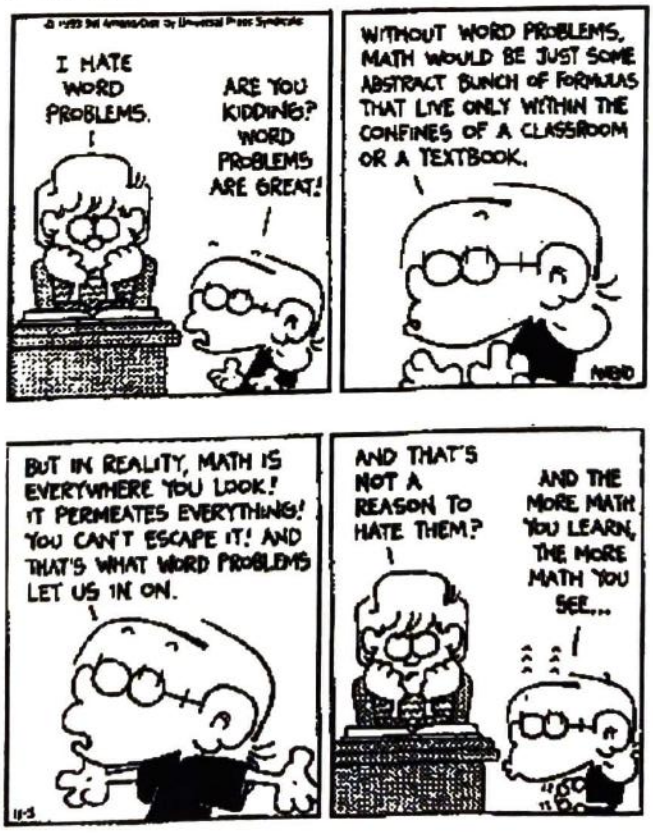
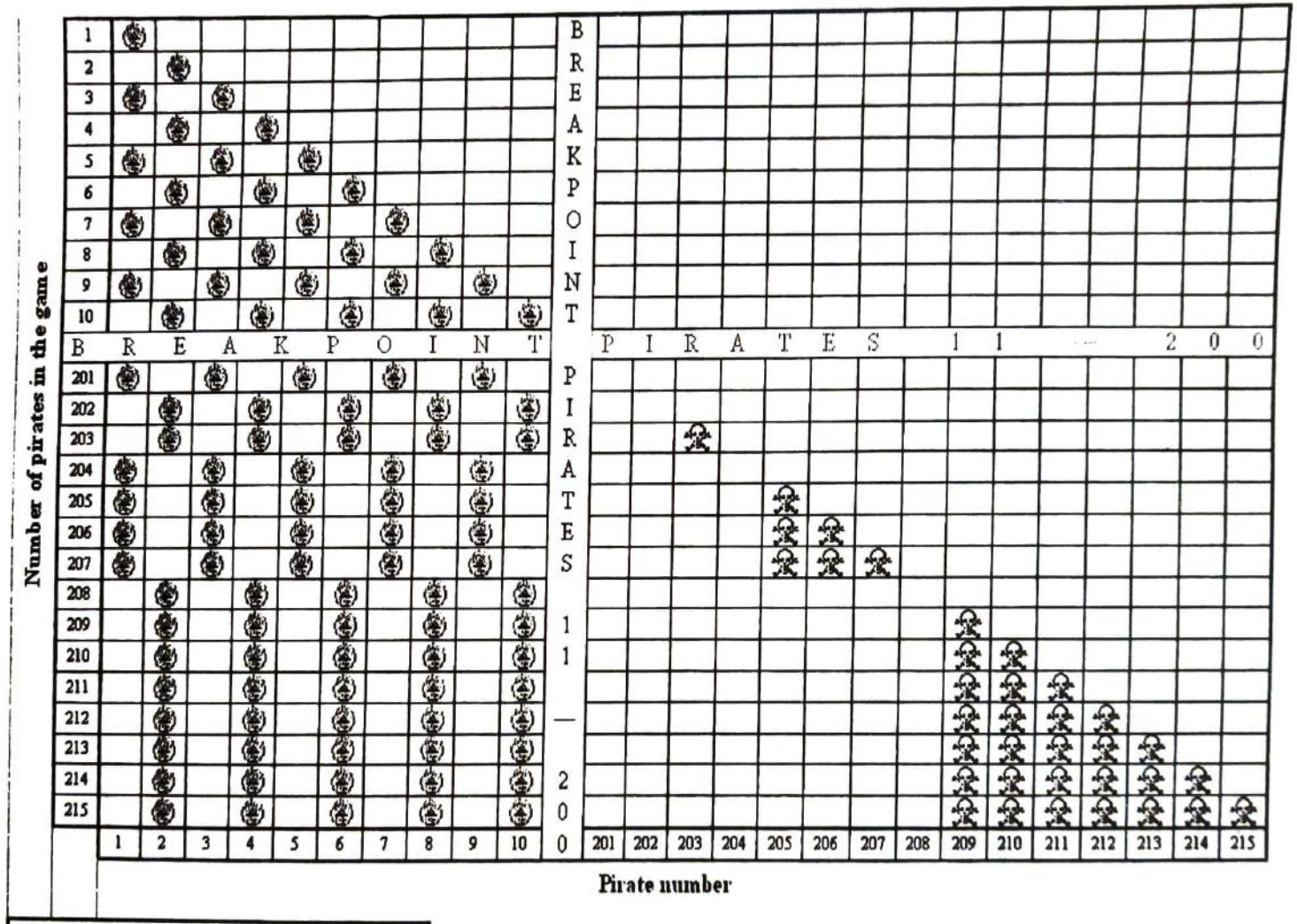


Figure 1



Legend	
	Pirate receives coin(s)
Empty cell	Pirate does not receive any coins
	Pirate is thrown overboard

Inverse Geometry

Shun Yu

1 Introduction

The origins of inversive geometry started with great geometer Apollonius of Perga (ca. 260-170 B.C.). Besides being famous for his work *Conics*, he defined the Apollonian curve, which proved to be the basis for inversive geometry. Jakob Steiner (1796-1863), a Swiss mathematician, later picked up this topic when he needed to solve a problem, called Steiner's Porism. From there, inversive geometry slowly developed. Today, inversive geometry is a branch of geometry that is used as a tool to simplified harder problems. In fact, an almost impossible geometry problem can be transformed to a very easy problem in a couple of inversions.

2 Apollonian Curves and Apollonius' Theorem

Perhaps the most basic definition in inversive geometry is the Apollonian curve. An Apollonian curve (denoted by $\mathbf{A_k}(A,B)$) is defined as the locus of points $\{P : PA = k \cdot PB\}$, where A and B are any two points on the plane and k is any nonzero positive constant.

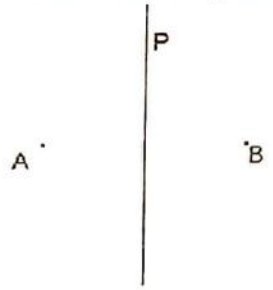
This definition of the Apollonian curve gives rise to Apollonius' Theorem, which states that for all $A_k(A,B)$:

(1) if $k = 1$, then $A_k(A,B)$ is a line. If $k \neq 1$, then $A_k(A,B)$ is a circle.

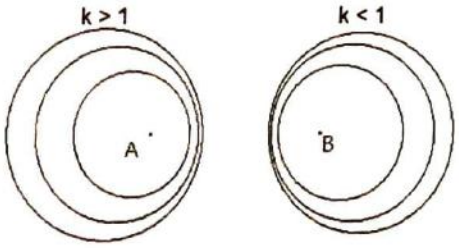
(2) If $A_k(A,B)$ is the circle C with center at O and radius r, then A, B and O are collinear, with A and B on the same side of O, and $OA \cdot OB = r^2$

(3) Conversely, if statement (2) is true, then C is A(A,B).

If $k = 1$, then $A(A,B) = \{P : PA = PB\}$, which is simply a line equidistant from A and B (or the perpendicular bisector of AB), like in figure (a).



(a)



(b)

For $k \neq 1$, $\mathbf{A_k}(A,B)$ can be any one (and only one) of the circles surrounding A or B. As k gets bigger, the $\mathbf{A_k}(A,B)$ gets smaller and closer to A. Similarly, as k gets smaller $\mathbf{A_k}(A,B)$ gets smaller but closer to B, as in figure (b). The reason I mentioned that $\mathbf{A_k}(A,B)$ can be one and only one of the circles is because the members of Apollonian curves are disjoint (having no elements in common) and unique. For example, lets take any two constants x and y. $\mathbf{A_x}(A,B) = \{P : PA = x \cdot PB\}$ and $\mathbf{A_y}(A,B) = \{P : PA = y \cdot PB\}$. Thus if P = P, $PA = x \cdot PB = y \cdot PB$ and therefore, $x = y$. Given any point P: P can only lie on one and only one of the circles (which also means that the circles never intersect).

The equation of the Apollonian curve can also be derived. Looking at figure 1, suppose A and B lie both on the x-axis and is distance a ($a \neq 0$) from the y-axis and C is the apollonian curve,

$A_k(A,B)$, where $k \neq 1$. Thus, the coordinates of A and B are $(-a,0)$ and $(a,0)$ respectively. If the point $P(x,y)$ lies on C, then it has to satisfy the equation, $PA = k \cdot PB$. Using the distance formula, we get:

$$\overline{PA} = k\overline{PB}$$

$$\sqrt{(x-a)^2+(y-0)^2} = k\sqrt{(x-(-a))^2+(y-0)^2}$$

$$(x-a)^2+y^2 = k^2[(x+a)^2+y^2]$$

$$k^2(x^2+2ax+a^2+y^2)-(x^2-2ax+a^2+y^2)=0$$

$$k^2(x^2+a^2+y^2)+2k^2ax-(x^2+a^2+y^2)+2ax=0$$

$$(k^2-1)(x^2+a^2+y^2)+2ax(k^2+1)=0$$

If $k = 1$, then the equation becomes $4ax = 0$ and since we are assuming, $a \neq 0$, we can divide both sides by $4a$ to get $x = 0$, which happen to be the line equidistant from A and B. If $k \neq 1$, then we can divide both sides by $(k^2 - 1)$ to get:

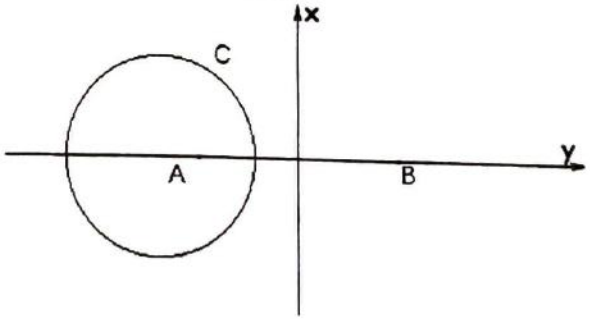
$$(x^2+a^2+y^2)+2ax\left(\frac{k^2+1}{k^2-1}\right)=0$$

$$x^2+a^2+y^2+2ax\left(\frac{k^2+1}{k^2-1}\right)+a^2\left(\frac{k^2+1}{k^2-1}\right)^2 = a^2\left(\frac{k^2+1}{k^2-1}\right)^2$$

$$\left[x^2+2ax\frac{k^2+1}{k^2-1}+a^2\left(\frac{k^2+1}{k^2-1}\right)^2\right]+y^2 = a^2\left(\frac{k^2+1}{k^2-1}\right)^2-a^2$$

$$\left[x+\frac{a(k^2+1)}{k^2-1}\right]^2+y^2 = \frac{4a^2k^2}{k^4-2k^2+1}$$

This is the equation of the circle C. It has the origin at $\left(-\frac{a(k^2+1)}{k^2-1}, 0\right)$ with $r^2 = \frac{4a^2k^2}{k^4-2k^2+1}$, where r is the radius of the circle C. Also notice that as k approaches 1, r approaches infinity since the denominator k^2-1 approaches 0. In inversive geometry, a line can be interpreted as a circle with an infinite radius. Hence, statement 1 is true.



(figure 1)

A, B and O are collinear because, according to the equation of circle C, A = $(-a,0)$, B = $(a,0)$, and O = $\left(-\frac{a(k^2+1)}{k^2-1}, 0\right)$. This means that all three points are on the x-axis and therefore are collinear. Hence, statement 2 is true.

Statement 3 is also true because OA = $\frac{a(k^2+1)}{k^2-1}+a$ and OB = $\frac{a(k^2+1)}{k^2-1}-a$. (OA)(OB) =

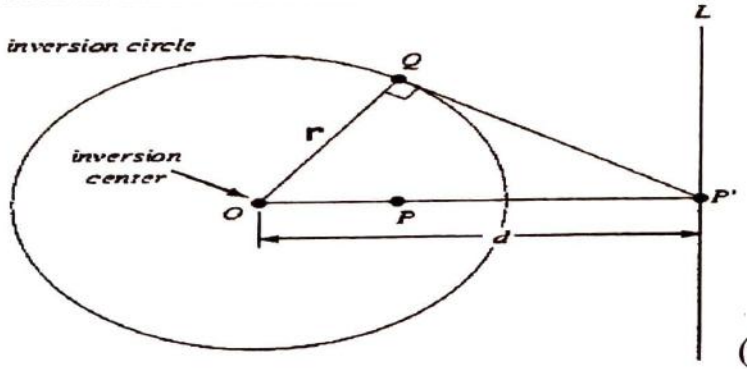
$a^2(\frac{k^2+1}{k^2-1})^2-a^2=\frac{4a^2k^2}{k^4-2k^2+1}=r^2$.

3 Inverse Points

If two points P and P' are collinear with a third point O and are located such that $(\overline{OP})(\overline{OP'})=k$ where k is any nonzero constant, P and P' are said to be inverses of each other with respect to O, called the center of inversion. If k is positive, P and P' are on the same side of O. If k is negative, P and P' are on opposite sides of O. The only time P = P' is when $(\overline{OP})(\overline{OP})=k$ or $\overline{OP}=\sqrt{k}$. The circle with center at O and a radius of $\sqrt{k}$ is called the circle of inversion. Since the radius is not imaginary, $k \geq 0$ and thus P and P' has to be on the same side of O, the center of inversion. Given any Euclidean plane, every point has an inverse point. The only exception to this is the center of inversion. If P = O, then OP = 0 and thus the equation $(\overline{OP})(\overline{OP'})=k$ cannot be true since $k \neq 0$. You can also notice that as $\overline{OP} \to \infty$, $\overline{OP'} \to 0$.

Looking back at the Apollonian curves and figure 1, we can notice that points A and B are inverse points with respect to the center (lets call it O) of circle C. That's because according to statement 2 of Apollonius' Theorem, if $A_k(A,B)$ is the circle C with center at O and radius r, then A, B and O are collinear, with A and B on the same side of O, and OA · OB = r^2. Suppose r = $\sqrt{k}$, then (OA)(OB) = $\sqrt{k}^2 = k$. Therefore, the two points A and B are collinear with a third point O and are located such that (OA)(OB) = k. Hence, A and B are inverse points with respect to the center of circle C. This connection allows us to some of the properties of Apollonian curves with inversion.

4 Inversion and the Inversion theorem



(figure 2)

The inversion function is defined to be a function that maps any point or locus of points to its inverse point(s). It is denoted as $i_c(P)$, where C is the circle of inversion. This value is equivalent to P', the inverse of P.

The Inversion Theorem states that given any circle with center O:

(1) If L is a line that passes through O, then L' = L.

(2) The inverse of a line not through O is a circle through O.

(3) The inverse of a circle not through O is a circle not through O.

Let P be any point of the line L. Since P' is also on the same line as P and O (definition of inverse points), P' lies on L. This is true for any point so all points P' lies on L. Since L' is the locus of all points P', L' = L. This proves the first statement true. The second and third statement can be proven later.

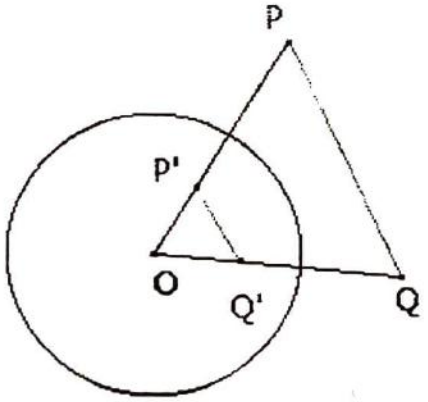
5 Basic Properties of Inversions

But before we prove that, let us first list some basic properties of inversions. The inversion function has order 2, which means that P'' = P. Looking back at figure 2, notice that it doesn't really matter which point is P or P'. The point inside the circle can be called P' and the point on line L can be called P. That is because multiplication is commutative, thus (OP)(OP') = (OP')(OP). Therefore, P can be the inverse of P' just as P' can be the inverse of P. This property allows us to use the converses of the statements in the Inversion Theorem. For example, if statement 2 is true, the inverse of a circle through O is a line not through O.

Another interesting property of inversion is that two pairs of inverse points are concyclic. If any four points lie on a circle, they are said to be concyclic. This will be proven later too.

6 The Equal Angles Theorem

The Equal Angles Theorem states that given circle C with center O and radius r, and points P' and Q' are inverses of P and Q with respect to C, respectively, (or denoted as $i_c(P) = P'$ and $i_c(Q) = Q'$) then $\angle QPO = \angle P'Q'O$ (Look at diagram below). That's because (OP)(OP') = r^2 and (OQ)(OQ') = r^2. Therefore, (OP)(OP') = (OQ)(OQ') or OP/OQ = OQ'/OP'. Since $\angle O = \angle O$, $\triangle OPQ \approx \triangle OQ'P'$ and thus $\angle QPO = \angle P'Q'O$.



(figure 3)

7 Applications of the Equal Angles Theorem

This theorem allows us to prove that two pairs of inverse points are concyclic. Looking at figure 3, if PQQ'P' is concyclic, then we must prove that either $\angle PP'Q' = 180° - \angle PQQ'$ or $\angle QPP' = 180° - \angle P'Q'Q$ is true (a property of concyclic quadrilaterals is that opposite angles add up to 180). Since Q' is on OQ:

$$\angle P'Q'O = 180° - \angle P'Q'Q.$$

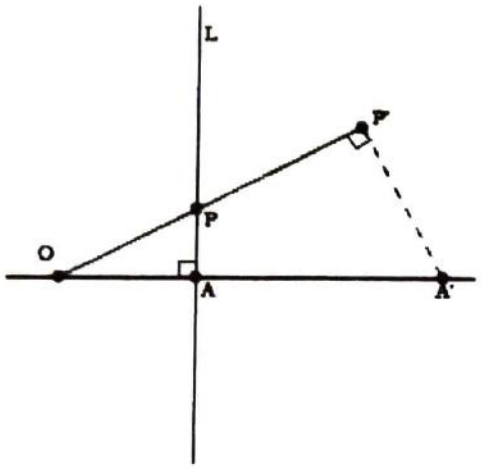
$$\angle QPO = \angle P'Q'O. \text{ (The Equal Angles Theorem)}$$

$$\angle QPO = 180° - \angle P'Q'Q.$$

$\square QPP' = 180° - \square P'Q'Q$ (Since $\square QPP' = \square QPO$)

And thus we have proved that two pairs of inverse points are concyclic.

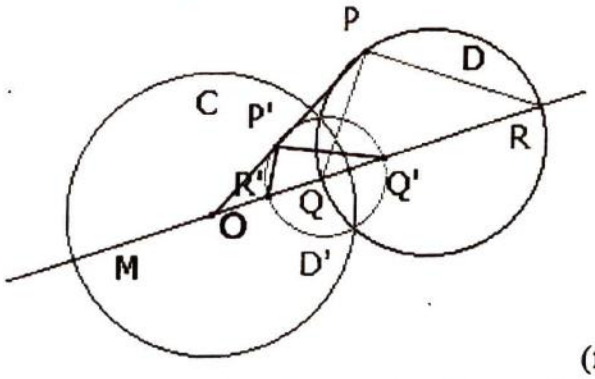
This property allows us to prove the second statement of the Inversion Theorem: The inverse of a line not through O is a circle through O.



(figure 4)

Looking at figure 4, let O be the center of inversion and A and P are two points on the given line L. Let A be situated such that OA is perpendicular to L. Let P be any point other than A. Therefore, $\square OAP = 90°$. Similarly, $\square PAA' = 90°$. Since we proved that two pairs of inverse points are concyclic, $\square PP'A' = 180° - \square PAA' = 180° - (90°) = 90°$. $\square PP'A'$ is a right angle. Thus, the locus of P' is a circle with diameter OA'. Also note that L is perpendicular to the diameter of the circle (this will be important in section 7).

This theorem also allows us to prove the third statements of the Inversion Theorem: The inverse of a circle not through O is a circle not through O.



(figure 5)

Let O be the center of circle C and M be the line passing through the centers of C and D. M intersects D at Q and R. Let P be any point on D except for Q and R. Since P is an angle on the semicircle, $\square P$ is a right angle. Inverse P, Q, and R to P', Q' and R' respectively.

$\square R'P'Q' = \square OP'Q' - OP'R'$ (Subtraction Postulate)

$\square R'P'Q' = \square OP'Q' - (\square OQP)$ (Equal Angles Theorem)

$\square R'P'Q' = (\square ORP) - \square OQP$ (Equal Angles Theorem)

$\square R'P'Q' = \square ORP - (180° - \square RQP)$ (Q is on OR)

$\square R'P'Q' = (\square ORP + \square RQP) - 180^\circ$

$\square R'P'Q' = (180^\circ - \square QPR) - 180^\circ$ (There is 180° in a triangle)

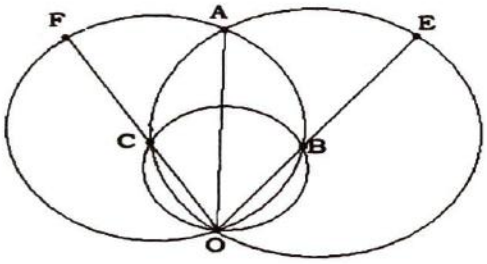
$\square R'P'Q' = -\square QPR^\circ$

$\square R'P'Q' = 90^\circ$ (Substitution and a -90° angle is still a right angle so it doesn't matter)

Thus, the locus of D' is a circle with diameter R'Q' and we have proved the Inversion Theorem.

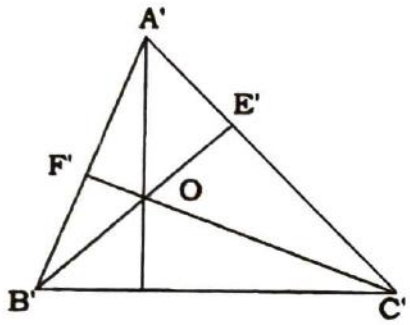
8 Applications of Inversion

Inversive geometry is a branch of mathematics. Alone, inversive geometry is not very interesting nor useful. It is more of a tool used in problem solving. At times, inversion can be a very powerful tool for solving otherwise unsolvable problems or for simplifying a problem. For example, the Inversion Theorem finally comes in handy for this following problem.



(figure 6)

Looking at the figure above. Given two circles that intersect at points O and A. The diameter OE of the circle OCAE intersects the other circle at point B. The diameter OF of the circle OBAF intersect the other circle at point C. Prove AO passes through the center of OBC. If you invert all the points with O as the center of inversion, then you get figure 7. The circle OBAF that passes though the points B, A, and F inverts to a line that passes through the points B', A', and F' (the converse of statement 2 of the Inversion Theorem). Similarly, The circle OCAE that passes though the points C, A, and E inverts to a line that passes through the points C', A', and E'. By the first statement of the Inversion Theorem, the line OCF inverts to the line OC'F' and the line OBE inverts to the line OB'E'. Referring back to section 6, we noted that (by the second statement of the Inversion Theorem) the line that passes through B', A', and F' is perpendicular to the diameter (FO) of the circle. Since OCF=OC'F', OC'F' is perpendicular to B'A'F'. Therefore, OC'F' is an altitude of the triangle A'B'C'. Similarly, we can show that OB'E' is an altitude of the triangle A'B'C'.

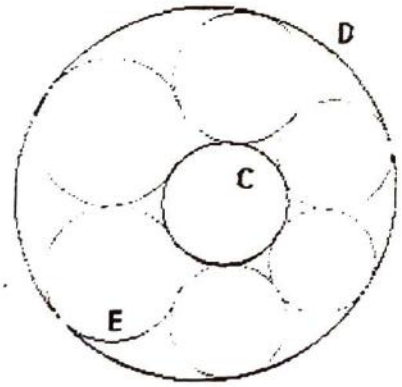


(figure 7)

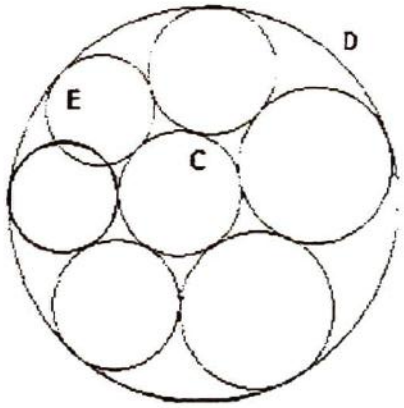
To prove that AO passes through the center of OBC, we only need to show that A'O is perpendicular to the line B'C', since inversion has order two (section 4). Since the three altitudes of a triangle are concurrent, A'O is an altitude to B'C' and therefore is perpendicular to the line B'C'. Therefore, we have proven our statement using inversive geometry. The proof, although possible, would have taken much longer without inversive geometry. And this is but one of the many things inversive geometry can do.

9 Steiner's Porism

Given any two circles, C and D, draw any circle E that is tangent to both. Draw another circle tangent to circles C, D, E. Next, draw another circle tangent to C, D, and the previous circle. Continue doing this until the last circle is tangent to C and D and is either tangent to E or intersects E. This sequence is called the Steiner Chain. Figures 8a and 8b displays the two possibilities. Both chains have six circles between C and D.



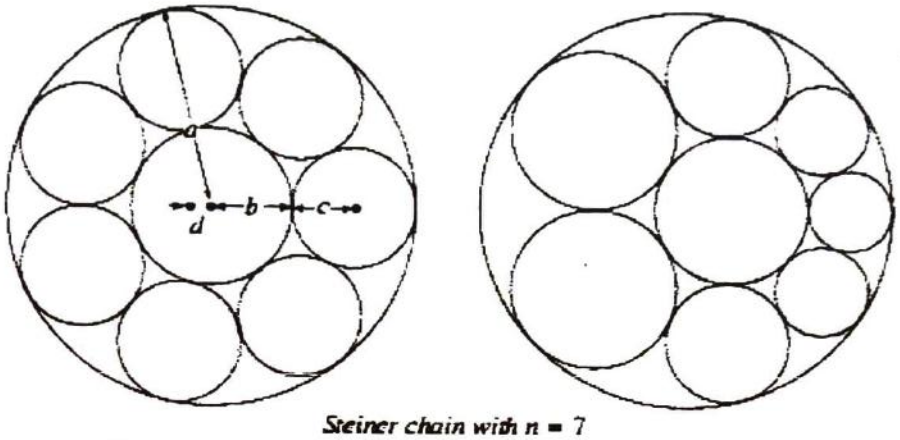
(figure 8a)



(figure 8b)

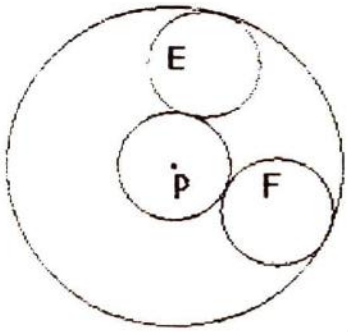
Steiner's Porism: Given any Steiner chain for any circle E and any circle F such that both are tangent to C and D, the number of circles formed (there is six in figure 8) is the same.

According to the Concentric Circles Theorem (the proof involves coaxal and orthogonal circles), given two non-intersecting circles, there is an inversion that maps the two circles to two concentric circles. Therefore, we can invert C and D into concentric circles. This is shown in figure 9 as figure 9b is inverted into 9a.



(figure 9a) After (figure 9b) Before

Looking at the inverted figure (next page), we can see that circle F is a mere rotation of E around P and thus the number of circles is the same.



(figure 10)

Bibliography

1. Altshiller-Court, Nathan. College Geometry. Johnson Publishing Company. USA. 1925
2. http://aleph0.clarku.edu/~djoyce/java/round/round1.html
3. http://www.cmis.brighton.ac.uk/staff/jt40/EM225/EM225_Inversive_geometry.pdf
4. http://www.maa.org/editorial/knot/Circle.html
5. http://www.cut-the-knot.org/Curriculum/Geometry/Apollonius.shtml#explanation
6. http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html
7. http://mathworld.wolfram.com/InversiveGeometry.html

figure a, figure b http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html
figure 1 http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html
figure 2 http://mathworld.wolfram.com/Inversion.html
figure 3 http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html
figure 5 http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html
figure 8a, figure 8b http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html
figure 9 http://mathworld.wolfram.com/SteinerChain.html
figure 10 http://www.maths.gla.ac.uk/~wws/cabripages/inversive/inversive0.html

The Mysteries of Infinity

Benjamin Lerner

1 Introduction

Infinity is defined as the number of elements in the set of natural numbers. Obviously, the first difficulty of dealing with infinity is figuring out just how to deal with infinity. What should it be treated as? Can it simply be treated as any other quantity?

First, let us consider the set of all natural numbers {1, 2, 3, ...} and the set of all perfect squares {1, 4, 9, ...}. Thinking about it one way, it is clear that every number isn't a perfect square and therefore there must be more natural numbers then perfect squares. However, it is also clear that every natural number is the square root of one perfect square; then there must be an equal number of perfect squares and natural numbers (Rucker, 1995). Although both of these statements seem logical, they are also contradictory. So the question arises: which one is correct?

Well, let's first see if the standard rules of addition and subtraction hold. Consider ∞-1. The basic concept behind this is that you are taking the set of all natural numbers {1,2,3,4,5...} and subtracting one element. Since it doesn't really matter which element it is, we can be left with the set {2,3,4,5...}. At first, it seems as if this set is smaller than it was before we took away a member; logically, it should be that way. However, it is clear that for every member in {1,2,3,4...}, there is a member in {2,3,4,5...}. Therefore, the number of elements in the two sets must be equal. This leads us to the conclusion that ∞-1=∞! Certainly, no matter what finite number of elements you take out from a set with an number of elements, there will still be just as many left. This holds true for addition as well: ∞+1=∞. What about multiplication and division? Well, consider ∞/2. This can be shown as the set of all odd numbers {1,3,5...} has had every other member of {1,2,3,4...} removed. Even so, we can still set up a one-to-one correspondence between the members of the two sets. Clearly, ∞/2=∞.

2 Size of Infinities

If $\infty+1 = 2\infty = \infty$, then the question arises: do all infinity sets have the same number of elements? For example, consider the set of integers, or {...-2,-1,0,1,2...}. Perhaps this set has more members than the set of natural numbers; after all, while natural numbers have a beginning, but no end, the set of integers has no beginning *and* no end... or does it? With a little rearranging, the set of integers can be made to look as follows: {0,-1,1,-2,2,-3...}. This can be put into a one-to-one correspondence with the natural numbers.

Then, let us consider the set of rational numbers. There are infinitely many fractions between any two natural numbers and thus, it should follow that there are *more* fractions than natural numbers. Even so, you can still write out the rational numbers in a logical order; this is all that is necessary for a one-to-one correspondence, as seen in figure 1.

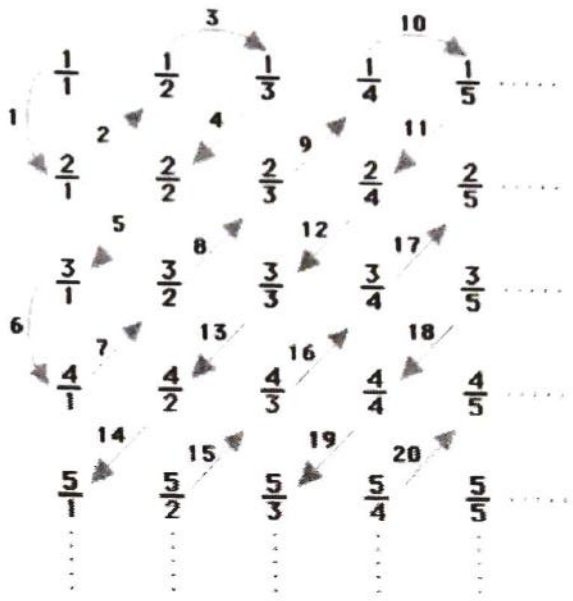


Figure 1[1]

Although this only includes positive rational numbers, the list can easily be expanded to account for all rational numbers. Faced with these infinite sets, it becomes tempting to say that all infinite sets have the same number of members. However, let us consider one further set: the set of all real numbers, which does *not* have the same number of members as the previously mentioned sets. For contradiction, let us assume that we have a list of all the real numbers, in one-to-one correspondence with the natural numbers. Without loss of generality, we can say that the list appears as follows:

1 – 3.14159…

2 – 2.7891011….

3 – 0.101001000100001…

.

.

.

n – 9842.245034534…

To show that this list does not contain all the real numbers, let us proceed as follows: take a real number greater than zero but less than one. Then, let us make the tenths digit of this number different from the tenths digits of the number which corresponds to 1 on our list; in this case, we may choose any digit other than 1; let's say 2. Let the hundredths digit of this number be different from the hundredths digit of the number that corresponds to 2 on our list; for example, let it be 9. Do the same thing for the thousandths place; let us take 2. Our number is, so far, 0.293… This number is definitely not the same as the ones in the first, second, and third positions on our list, since its tenths digit is different from that of the first number, its hundredths digit is different from that of the second number, and its thousandths digit is different from that of the third. If, in general, we let the digit in the 10^{-n} digits place be different from the digit in the same place of the real number in the n^{th} position on our

[1] This diagram may be found at http://scidiv.bcc.ctc.edu/Math/Rat=Nat.html

list, the number that will be produced after going through all n natural numbers will not be on the list[2]. Therefore, we have proven that, although both sets have infinitely many members, the set of all real numbers has "more" members then the set of all natural numbers. (Counting to Infinity)

3 The Wonders of ℵ

ℵ, pronounced "aleph", is a character in the Hebrew alphabet. It is also the symbol used by Georg Cantor to signify what he called transfinite[3] numbers. The first of these, $\aleph_0$ (pronounced "aleph null"), is defined as the number of elements in the set of all natural numbers. $\aleph_1$ is the number of members of the next highest infinite set, or the set of all real numbers, and so on, with $\aleph_2$, $\aleph_3$, etc. It should be noted that there are precisely $\aleph_0$ different transfinite numbers. Additionally, $\aleph_0$ is referred to as a countable infinity; everything else is referred to as uncountable.

How can infinity be "countable"? The concept of one infinity being greater than another is, at first glance, completely ridiculous. After all, if infinity is truly unlimited, how can there be "more" infinity? There are several ways to define this, none of which work particularly well. For example, it can be defined as follows: a set that is "more" infinite than another set will, no matter how much of the second set you write out, always have infinitely more members. However, this definition is ridiculous; the same can be said of those sets in the opposite order.

I believe that, in order to understand this properly, one must refer to the concept of "potential" infinity. When applied to natural numbers, this concept becomes almost worthless. However, when applied to countable and uncountable infinities, it is a great help.

Although it is clear that, when comparing two infinities one cannot possibly be more than the other, if you consider potential infinite, the concept of being "more" infinite becomes graspable. Essentially, if you could somehow write out the entire infinite set of the set with "less" infinite members, then the set with "more" members will still have an infinite number of elements left. Then, the definition becomes clear; a set of size $\aleph_1$ is not actually greater then a set of size $\aleph_0$; it is only potentially greater.

4 Conclusion

Infinity is a mind-boggling concept. To most people, it is something which induces fear, a fear of something far, far greater than yourself. To others, infinity is a wondrous, elusive concept; something utterly impossible to capture. Through my investigation of infinity, I have effectively answered all my questions, as well as some I didn't even know I had; but, instead of leading to contentment, it has lead to the formulation of a number of new questions; it is time I set forth to investigate them.

Sources

Counting to infinity. (n.d.). retrieved Dec. 3, 2004, from http://scidiv.bcc.ctc.edu/Math/infinity.html.

[2] Although it is true that this number could simply be added to the list corresponding to n+1, it is important to note that we had assumed we already had a full list when we started. Furthermore, even if the number is added, there will always be some other number that is *not* in the list.

[3] Transfinite numbers are so named because Cantor did not consider them truly infinity, as if they were truly infinity then there would be no numbers surpassing them.

Devlin, K. (2001). Devlin's angle: How many real numbers are there?. retrieved Dec. 3, 2004, from MAA Online Web site: http://www.maa.org/devlin/devlin_6_01.html.

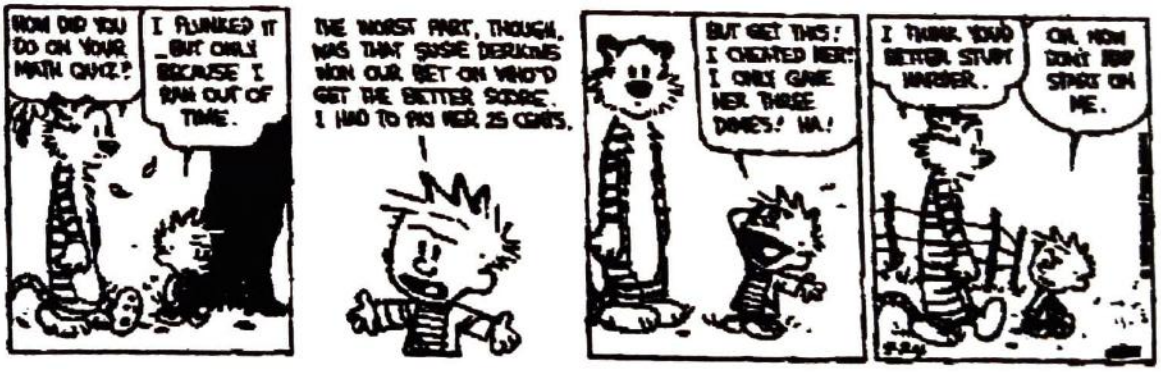
O'Connor, J. J., & Robertson, E. F. (1996). A history of set theory. retrieved Dec. 3, 2004, from Web site: http://www-gap.dcs.st-and.ac.uk/~history/HistTopics/Beginnings_of_set_theory.html

O'Connor, J. J.,& Robertson, E. F. (2002). Infinity. retrieved Dec. 3, 2004, from http://www-gap.dcs.st-and.ac.uk/~history/HistTopics/Infinity.html.

O'Connor, J. J., & Robertson, E. F. (1999). Zeno of Elea. retrieved Dec. 3, 2004, from Web site: http://www-gap.dcs.st-and.ac.uk/~history/Mathematicians/Zeno_of_Elea.html

Retrieved December 13, 2004 from the World Wide Web: http://homepages.wmich.edu/~korista/web-images/accretion_ncstate.jpg

Rucker, Rudy. Infinity and the Mind: The Science and Philosophy of the Infinity. 1st ed. Princeton: Princeton University Press, 1995.



An Introduction to the Surreal Numbers

Mark Spindler

Throughout our limited mathematical careers, we focus primarily on the mathematics of the real number system. It contains many numbers we are intuitively familiar with, like 0 and 1, and many numbers taught to us like ¯3 and π. This choice of the real number system over any other number system is somewhat arbitrary, as there are other choices which share every property we, as high school students, would need numbers to satisfy. One such option is the surreal numbers, invented by John Conway.

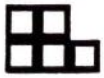
A surreal number is defined as an ordered pair of sets of nothing except previously created surreal numbers such that no number in the right set is less than or equal to any number in the left set. A surreal number x is less than or equal to another surreal number y if and only if y is less than or equal to no member of x's left set, and no member of y's right set is less than or equal to x. Due to the self referential (indeed, recursive) nature of these definitions, it may seem impossible to exhibit a surreal number. However, it turns out that ({},{}) is a valid surreal number.

Is ({},{}) an ordered pair of sets containing nothing except previously created surreal numbers? Yes, the sets contain nothing at all. Does ({},{}) satisfy the other condition? Yes, no number in the right set has any property at all. We can call this newfound surreal number 0 (justification for this will come later, when we define operations on surreal numbers). We have finally created a surreal number. We can now construct new ones by putting this in the sets. ({0},{}),({},{0}), and ({0},{0}) might all be surreal numbers. However, 0≤0 (since the two "no member" conditions must be satisfied, since the sets have no members at all), and so ({0},{0}) is not a surreal number at all. The other two are valid surreal numbers and can be called 1 and ¯1 respectively. By putting the numbers in this new list into sets, we can create more and more surreal numbers, eventually generating replacements for all of the real numbers and many more. The extra numbers that result from such a seemingly simple definition fall into two major categories: infinities and infinitesimals. An infinity is a surreal number that is greater than (not less than or equal to) any surreal number that represents a corresponding real. An infinitesimal is a surreal number greater than ({},{}) (the surreal equivalent of zero) but less than any positive surreal that represents a corresponding real.

Numbers are useless without operations defined on them. The sum of two surreal numbers a and b is defined as follows $a+b$=((the set containing every sum of b and an element of a's left set) ∩ (the set containing every sum of a and an element of b's left set),(the set containing every sum of b and an element of a's right set) ∩ (the set containing every sum of a and an element of b's right set)). Once again, we have a recursive definition. Since 0=({},{}) we can see that $0+x=x$, thus confirming our naming of 0. Similarly, 1+¯1=0, etc. Similarly, we can create good names for other numbers and justify them by using this addition rule. However, all of our names might be off by a constant of multiplication, and at this point we have no way of knowing. You can rest assured that there are

definitions of multiplication, multiplicative inverses, and a mapping from reals to their surreal equivalents, that satisfy every property that we'd like them to have, such as the product of two surreal equivalents of reals is the surreal equivalent of the product of the reals, etc. However, these definitions are a bit too complicated to present in this article. Luckily, surreal numbers have another use, other than that they are a substitute for the reals, that we are ready to look at. They are extremely powerful in game theory.

Consider a game with no hidden information in which two players, "Left" and "Right" take turns. If both players play optimally, either Left will definitely win, Right will definitely win, the player who goes first will definitely win, or the player who goes second will definitely win. Surreal numbers can help us figure out what will happen in a given game. One classic game of this type is stopgate. A game of stopgate is played on one or more collections of squares that are orthogonally connected. Players take turns placing dominoes on the board until one person cannot place one (on squares but not overlapping any other domino) on their turn. That person loses. Left places dominoes horizontally and Right places them vertically. The simplest possible game of stopgate is played on no board at all. There is no room to place a domino in any direction, and so the first person to go loses. Another way of saying this is that each player has no moves, and so if we made an ordered pair of sets of all of the moves that the players can make, it would look like this ({},{}). This looks just like the surreal number 0. Another game is played on the following board:  . In this game of stopgate, there is still no room to place a domino in any direction, and so we are left with the same first person loses ({},{}) situation. A more interesting game is  . In this game, if Left goes, the available board turns into no board at all (0), and Right has no moves at all. This is represented by ({0},{})=1. Notice that in this case, no matter who goes first, Left wins, and this game has a positive value (1). Similarly,  =({},{0})=-1, and Right always wins.  =({0},{})=1 because either of the moves for Left turn it into  and Right can't move at all. Now consider  . In this game, Right can turn it into  =0, and so can Left, so this game is ({0},{0}). Note that this is not a surreal number, and so we can't use anything like a real number to represent it. These game theory numbers that don't correspond to any surreal number are called fuzzy numbers, and ({0},{0})=*. This game with a value of * is such that whoever goes first wins.

This pattern of the number representing what happens under optimal play continues, i.e. 0=first player loses, positive=left wins, negative=right wins, fuzzy=first player wins.  is another fuzzy game, since each player will block the placement of the other direction of domino when they move.  is the first game where surreal numbers can tell you things about these types of games that you couldn't figure out as easily otherwise. Right can turn the board into  =* or  . This second position contains two disjoint boards. It is a theorem that the value of a bunch of disjoint (completely

separate) mini-games is the sum (as defined for surreal numbers) of the values of the mini-games, so  =-1+0=-1. Left can turn the game into  =1+0=1 or =1 so this game is ({1},{*,-1}). Since, ignoring the *, this number is fuzzy anyway, this game has a fuzzy value and hence the first player wins (assuming optimal play). This method can be used to analyze any game of stopgate. A final note:  =({0+0=0,1},{})=2, which indicates that when is combined with other pieces, it will help Left more than a mere . This is why   is won by Left, because 2-1=1>0.

Math Jokes

•A topologist is a person who doesn't know the difference between a coffee cup and a doughnut.

•To mathematicians, solutions mean finding the answers. But to chemists, solutions are things that are still all mixed up.

•Q: Why do you rarely find mathematicians spending time at the beach?
A: Because they have sine and cosine to get a tan and don't need the sun!

•Mathematicians never die - they only loose some of their functions.

•"What happened to your girlfriend, that really cute math student?"
"She no longer is my girlfriend. I caught her cheating on me."
"I don't believe that she cheated on you!"
"Well, a couple of nights ago I called her on the phone, and she told me that she was in bed wrestling with three unknowns..."

Complete Information Algorithms in Cake Cutting

Zuoyu Tao

April 15, 2005

1 Introduction

The problem of the division of a cake was first formulated by the Polish school of mathematicians in the 1940's. Especially in recent times, this problem has drawn the attention of mathematicians and ecnomists who have been interested both by the elegant (and deceptive) simplicity of this problem and by its possible applications in resolving conflicts, disputes, and reaching a negotiated settlement.

In this paper, we discuss cake-cutting using complete information, a recent idea not yet explored in existing literature. We begin with an overview of the cake-cutting algorithms in the literature, followed by our new general n-person procedure in the context of complete information. We give a proof for it in subsequent sections and conclude with several possible applications of this algorithm and possible future directions for study.

2 Background

Cake, in the sense of this problem, represents any connected, continuous, divisible good that n players need to divide amongst themselves evenly. However, the player's valuations of parts of the cake differ. For example, if the cake consists of a mixture of chocolate and vanilla, one player may detest the chocolate, and another may love it. Therefore, the first player will judge the chocolate portion of the cake to be nearly worthless, and the second player may well judge it to contain nearly the entire value of the cake. The challenge is to

find an algorithm, or a set of steps players can follow, that guarantees each player a piece whose value to him is at least $1/n$ of the value of the cake. We call this type of division **fair division**. What complicates matters, however, is that we cannot be certain that the players will always be honest about their valuations. For example, if the algorithm tells a player to cut a piece he considers to be worth $\frac{1}{3}$ of the cake, the player could be dishonest by cutting a bigger or smaller piece instead. The algorithm should make sure that players who lie run the risk of obtaining less than $1/n$ of the cake.

To make the problem mathematically rigorous, we need to introduce several conditions. First, we assume that all players are completely risk-averse, and follow a **maximin** strategy: they will not try to manipulate the algorithm for a larger piece if there is a chance they will get a smaller piece instead. Secondly, we assume that the valuation functions for each player (the valuation function is the function for which its integral over a certain interval equals the value the player assigns to that interval of cake) is continuous and greater than zero at all times. The integral of the valuation function over the entire cake is 1. Thirdly, the values of disjoint pieces to a player combine additively. Fourthly, for simplicity we shall model the cake as a straight line to be cut into n pieces. Finally, we assume that the cake is nonatomic, or infinitely divisible.

Based on these assumptions, there is a simple, age old, algorithm for the two player case, called "cut-and-choose". One player cuts the cake into what he considers to be two portions of equal value. The other player chooses one of the two pieces, with the first player receiving the other. Obviously, the first player, the cutter, gets a piece he values at 1/2 of the cake, because he has cut both portions to be equal. The second player also must get a piece *he* values at 1/2 of the cake, since if the second player considers one portion to be less than 1/2, he will choose the other portion, which must be greater than 1/2 by the additivity of valuations. Neither player can get a larger piece by being dishonest. If the cutter cuts the cake to of unequal value (to him), he has no guarantee that he will receive the larger piece.

There is an extension of this algorithm to n players [4]. A disinterested party (call him the referee) moves a knife across the cake from left to right. When any of the players considers the portion to the left of the knife to have value $1/n$, he calls "cut". The referee cuts the

cake at that point and gives the leftmost piece to that player, who drops out of the game. The procedure then continues with the remaining $n-1$ players. It is easy to check that this algorithm gives a fair division of the cake and it, too, cannot be manipulated by the players.

However, even if the division is fair, players may be envious of others. If every player P_i receives piece M_i, then we say the division was **envy-free** if and only if for every i such that $1 \leq i \leq n$ and for every $k \in i$, $\mu_{P_i}(M_k) \leq \mu_{P_i}(M_i)$ where $\mu_{P_k}(M_j)$ is player P_k's valuation of piece M_j. In other words, a player will be envious if and only if he values any other piece to be more than his own, and the division will be envy-free when no player is envious (note that such a division implies a fair division). It has been proved that there are cut points that result in an envy free division for any number of players [5], but finding an algorithm to get this is considerably harder than for fair division. The two player cut-and-choose is envy free but the Dubins-Spanier procedure is not. The first $n-2$ players will very likely be envious, unless all the players' assigns values in exactly the same way.

Some envy-free three player algorithms exist, but only two are able to accomplish envy free division with minimal (two) cuts. One example is the three player "squeezing procedure" due to Barbanel and Brams [1]. For four players, there also exists envy-free algorithms [3], but they are much more complicated and no minimal cut (ie. 3 cuts) algorithm for four players is known to exist. A general n-player algorithm also exists [2], but there is no known bound on the (finite) number of cuts it requires. It could take n cuts, or a million cuts. Also, the algorithm rules are almost three pages long, involving many complicated subcases. These traits make it impractical for actual real world situations.

As an aside, and also as another reason to pursue minimal cake-cutting algorithms: an interesting property of minimal-cut procedures is that they will automatically be Pareto-optimal, that is, no player can do better by following another procedure unless the other players do worse. This is a very desirable property in the real world, where we often wish to allocate resources as efficiently as possible. Minimal-cut algorithms, though, are even harder to find than regular envy-free algorithms.

3 Cake-Cutting with Complete Information

In the existing literature, cake-cutting has so far been discussed with incomplete information. No player knows the valuations of the other players beyond that revealed through the choices another player makes. However, to create a bounded, $n-1$ cut envy-free algorithm, Barbanel and Brams have conjectured that it may be necessary for each player to have complete information about everyone else's valuations, and that this is common knowledge [1].

Even if players have complete information of what the others want, they do not trust another to cut the cake in an envy-free manner, so an algorithm specifying which players may cut and rules regarding the partitioning of the pieces is still required. We also disregard the uninteresting trivial algorithm, which basically lets one player cut for the rest.

Other assumptions we make are much the same as in regular cake-cutting. All players are assumed to be rational, follow a maximin strategy, and are risk-averse.

Given these assumptions, we can now introduce a general, minimal-cut, cake-cutting algorithm that I discovered, which answers Barbanel and Brams's conjecture partially in the affirmative.

4 The General n-Person Discrete Procedure

The discrete procedure allows a player to use only one cut to cut any connected piece from the cake. In effect, he must cut an "endpiece", or a piece on either end of the cake. This gives an envy free solution using $n-1$ cuts, but it does need a particular ordering of players.

The discrete procedure is as follows:

1. An ordering of players is determined (this will be discussed more later)
2. The first player in the order cuts a piece from the cake. This can be any piece, but can utilize at most one cut.
3. The player offers the piece to one player. If that player accepts, then that player takes the piece. If that player declines, then that player is moved to the very end of the ordering. The piece is then offered to other players. If no other player accepts, then the piece will be destroyed [1]and the first player is dropped from the game. Otherwise,

[1]This penalty is mostly to discourage "bad", or deliberately unacceptable offers: we will show that our algorithm gives an envy-free division with at least one permutation of players, and thus a piece need never actually be destroyed

the piece will be given to the player who accepts.

4. A player can *object* to an offer. In this case, the objecting player will be willing to take a smaller piece than the one offered. This is to eliminate ties and ensure a minimum possible offer is always made.
5. The player who gets the piece drops out of the game, and now the next player in the ordering (eliminating the player who gets the piece) repeats steps 1–3.
6. Steps 1–4 continue until there are two players left. At this point, the two players use the cut-and-choose algorithm (using only one cut) to split the remaining cake.

5 Proofs

This algorithm works for at least one ordering (permutation) of players. We offer a three person proof first.

5.1 A Three Player Proof

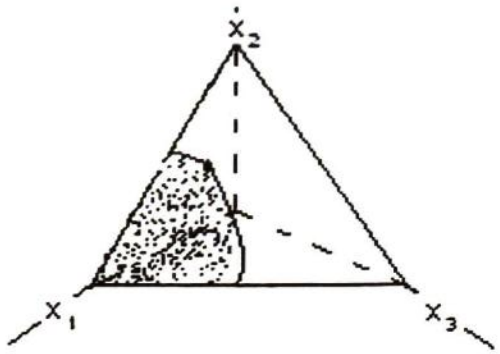
Consider the line representation of a cake, and consider a partition of this cake into three disjoint sets using only two cuts. Label the partitions from left to right x_1, x_2, and x_3. The *physical* sizes, as distinct from player valuations, of the partitions must sum up to 1, by the cake cutting assumptions we mentioned earlier. Notice that in three dimensions (labeled with axes x_1, x_2, and x_3) the equation $x_1 + x_2 + x_3 = 1$ produces a plane. Every point on this plane in the first octant is a possible partition of the cake, which we will call **cut-sets**. The surface of the plane in the first octant is also the 2-simplex, or equilateral triangle. Let us also introduce the useful notation that (a, b, c) refers to the cut-set where the physical size of $x_1 = a, x_2 = b$, and $x_3 = c$.

Now consider the ordering of players $\underline{ABC}$ without loss of generality. There is a unique cut-set $B_{\frac{1}{3}}$ on the 2-simplex where $\mu_B(x_1) = \mu_B(x_2) = \mu_B(x_3) = 1/3$. Suppose that player A wishes to offer a piece x_1 to B. B will accept x_1 if and only if $\mu_B(x_2) \leq \mu_B(x_1)$ and $\mu_B(x_3) \leq \mu_B(x_1)$[2]. As shown in the figure on the next page, the former condition can be represented by a curve from $B_{\frac{1}{3}}$ to a cut-set on the line connecting cut-set (1,0,0) with (0,1,0) (call this line $\overline{x_1x_3}$). At this cut-set (call it $B_{\frac{1}{2}x_1x_3}$), $\mu_B(x_1) = \mu_B(x_3) = 1/2$. At every point

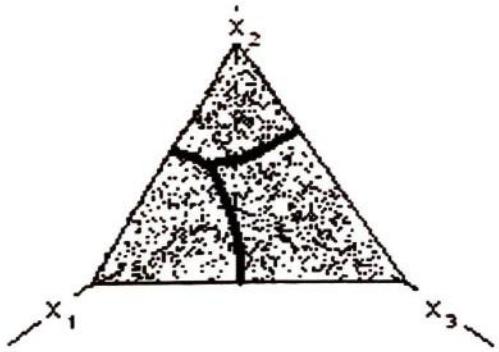
[2]In the three player case, these are the conditions for B to be envy-free

along the curve, $\mu_B(x_1) = \mu_B(x_3)$. Call this curve $\varphi^B_{x_1x_3}$. Similarly, to satisfy the latter condition we can draw a curve from $B_{\frac{1}{3}}$ to $B_{\frac{1}{2}x_1x_2}$ on the line $\overline{x_1x_2}$ connecting cut-set (1,0,0) with (0,0,1). Call this curve $\varphi^B_{x_1x_2}$.

Referring to the figure, we see that any cut-set in the shaded region (between the two curves) is acceptable for player B if B receives x_1. Let us define the "shaded area" of x_i^P to

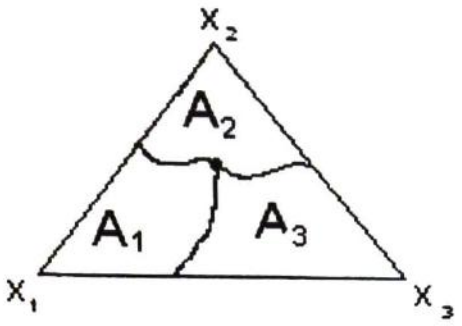


mean the region between the curves $\varphi^P_{x_ix_j}$ and $\varphi^P_{x_ix_k}$, where $i \in \{1,2,3\}$, $j = i+1 \mod 3$, $k = i+2 \mod 3$, and P is an arbitrary player. We see that the union of all "acceptable", or shaded areas for a specific P, covers the entire 2-simplex but do not intersect (see figure).



Label the shaded areas A_1, A_2, and A_3 by the vertex which bounds each one (see figure). Now label each cut-set in the region by the name of the region: for example, a cut-set in region A_1 would be labeled A_1. Cut-sets on a curve, such as $\varphi^A_{x_2x_3}$, will have the labels of the regions it borders. For example, a cut-set on $\varphi^A_{x_2x_3}$ will have labels A_2 and A_3 and the cut-set $A_{\frac{1}{3}}$ will have labels A_1, A_2, and A_3. This is illustrated in the figure below.

Now consider the shaded areas of B and C. Label them in the same manner as we did for



player A, except in the form B_1, B_2 and B_3 and C_1, C_2, C_3 respectively. Label the cut-sets in the shaded areas similarly too. Do the same with the curves $\varphi^B_{x_1x_2}$, $\varphi^B_{x_1x_3}$, $\varphi^C_{x_2x_3}$, $\varphi^C_{x_1x_2}$, $\varphi^C_{x_1x_3}$, and $\varphi^C_{x_2x_3}$.

These curves $\varphi^A_{x_1x_2}$, $\varphi^A_{x_1x_3}$, $\varphi^A_{x_2x_3}$, $\varphi^B_{x_1x_2}$, $\varphi^B_{x_2x_3}$, $\varphi^B_{x_1x_3}$, $\varphi^C_{x_1x_2}$, $\varphi^C_{x_1x_3}$, and $\varphi^C_{x_2x_3}$ divide the 2-simplex into a number of regions (some "regions" could be single points). We claim that there is a non-empty 2D region (call it region Ξ) where cut-sets are labeled A_i, B_j, and C_k, where $i \neq j \neq k$.

This follows from the proof of envy-freeness for n players with $n-1$ cuts given by Ichiisi and Idzik [5]. It is easy to see that the existence of our region is necessary and sufficient condition for the existence of an envy-free cut-set (see appendix **A.5** for more details).

Moreover, the region must only be bounded by the curves of the form $\varphi^P_{x_ix_j}$ and not by any edges, where $i, j \in (1, 2, 3)$ and P is an arbitrary player. This is simple to prove: at any edge some piece x_k will have value 0, so no player can prefer that piece, violating our assumption that in that region everyone is envy-freeness.

Two of the players prefer an "endpiece" (either piece x_1 or piece x_3) for any cut-set in region Ξ. Choose one player randomly, say player C, and assume without loss of generality that he prefers piece x_3. Now, we can model what happens if another player offers C piece x_3

For a fixed x_3 (or any endpiece for that matter), the set of all possible cut-sets is a 1-simplex, or line. To picture this, we can simply take the intersection of the plane $x_3 = c$ with our 2-simplex, where c is a constant. Obviously, this intersection is a line parallel to line $\overline{x_1x_2}$. On this 1-simplex, we can see that the set of "acceptable" cut-sets for B and A is

a 1−D region (call it λ) bounded by $\mu_A(x_2) = \mu_A(x_1)$ and $\mu_B(x_2) = \mu_B(x_1)$ [3]. However, on the 2-simplex these curves are simply $\varphi^B_{x_1x_2}$ and $\varphi^A_{x_1x_2}$.

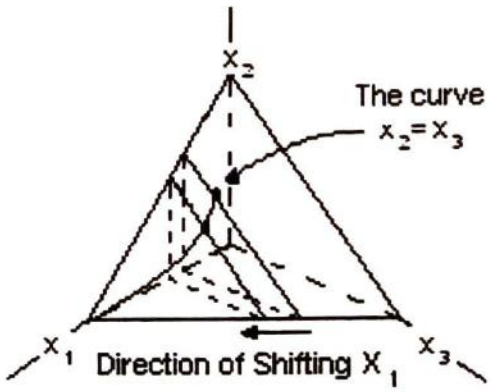
We introduce a lemma here that we use quite frequently.

Lemma 1. *Using this algorithm, players will always choose a cut-set at the boundary curves.*

Proof: This is follows from the fact that players pursue a maximin strategy, and that objections from players force a minimum possible offer. If a cut-set is inside an envy-free region, that implies that it is not a minimal offer, because we can reduce the size of a piece offered to any player and still get an envy-free solution. That is not possible only at a cut-set on the boundary curves. □

Moreover, the curves $\varphi^B_{x_1x_2}$ and $\varphi^A_{x_1x_2}$ help to bound the region Ξ too (see appendix **A.4**). Certainly, $\Xi \in \lambda$ for all values of x_3 because the conditions for a cut-set to be in λ is weaker than the condition for a cut-set to be in Ξ. A cut-set is an element of λ if it has labels A_i and B_j, where $i \neq j$. However, for a cut-set to be an element of Ξ it must also have label C_k.

We can now model what happens with increasing offers of x_3 to.C. Begin by considering when $x_3 = 0$ in terms of physical size. The 1-simplex will be $\overline{x_1x_2}$, and λ will be the region between $B_{\frac{1}{2}x_1x_2}$ and $A_{\frac{1}{2}x_1x_2}$. With increasing x_3 the 1-simplex "shifts" toward the vertex x_3, as shown in the figure on the next page.



Because $\Xi \in \lambda$ and they share a boundary, for a certain permutation of players somewhere there will be a point where a cut-set ι is on that shared boundary (in λ) and also $\iota \in \Xi$.

[3] Also let λ designate the region in the 2-simplex where cut-sets are labeled A_i and B_j, where $i \neq j$.

This will be the cut-set generated by our algorithm, and it is envy-free since it is in Ξ. Note that in our example, if player B offers to player C, then player A will choose a cut-set on (in reality, near) the bounding curve $\varphi^B_{x_1x_2}$ (see appendix **A.2** for details). Also note that for bigger offers of x_3, the cut-set will change, but will remain on the boundary curve $\varphi^B_{x_1x_2}$.

Note that in our above proof we assumed that B's bounding curve was $\varphi^B_{x_1x_2}$. It is possible that it is in fact $\varphi^B_{x_1x_3}$ which could create problems (see appendix **A.6**). However, because at least one of the bounding curves must be $\varphi^P_{x_1x_2}$ by the pigeonhole principle, we can let that P be the player who offers. The important thing is that our algorithm works for at least one, and quite possibly more, ordering of players.

Lemma 2. *Suppose that the algorithm can generate a cut-set (call it ι) on a boundary curve (call it φ) and ι is not the intersection of two such curves*[4]*. Also suppose ι is in Ξ, the envy-free region. Then every cut-set in $\Xi \cap \varphi$ can be generated by our algorithm.*

Proof: The proof of this lemma is simple in the three player case. We have seen how increasing the size of an offer, say piece x_k also implies a "shifting" of the final cut-set on a boundary curve $\varphi^P_{x_ix_j}$ (see above discussion on shifting of the 1-simplex). Therefore, a sufficiently large offer will allow for the cut-set to be anywhere on the boundary curve, as long as that cut-set is also in Ξ. The proof for the four player case depends first on the proof of the existence of an envy-free cut-set generated by our algorithm. However, once we prove the existence of an envy-free cut-set for four players, the proof of the lemma follows the same argument as does our three player proof of the lemma. □

5.2 Four Players and More

Just as the solution space (possible cut-sets) for two players is a 1-simplex and for three players is a 2-simplex, the solution space for n-players is an $n-1$-simplex (see appendix **A.3** for details). In the case of four players, it would be a tetrahedron.

Let us introduce the notation $(a, b, c, d)_P$ to define a cut-set in terms of a specific player's valuations, where $a = \mu_P(x_1)$, $b = \mu_P(x_2)$, $c = \mu_P(x_3)$ and $d = \mu_P(x_1)$ and where P is an

[4]If ι were on the intersection of the curves, this lemma applies to at least one of them

arbitrary player. Also, define $P_{\frac{1}{4}}$ to be $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})_P$

First, we examine the three dimensional analogue of the "shaded area" discussed earlier for the three player case. Consider the case of player A offering piece x_1 to player B. In this case, the analogue of the "shaded area" of x_1 would be the "shaded volume". It will be bounded by the three 2D surfaces $x_1 = x_2$, $x_1 = x_3$, and $x_1 = x_4$, with some restrictions. The surfaces will intersect at three curves, $x_1 = x_2 = x_3$, $x_1 = x_2 = x_4$ and $x_1 = x_3 = x_4$, and those will be part of the bounds for the shaded volume of x_1.

We can prove that the union of the shaded areas of x_1, x_2, x_3 and x_4 cover the entire 3-simplex by examining the surfaces bounding the shaded areas of x_2, x_3 and x_4. It is a very similar argument to the three person case.

Now consider $A_{\frac{1}{4}}$. Draw the three dimensional analogue of $\varphi^A_{x_1x_2}$, which would be $\mu_A(x_1) = \mu_A(x_2)$ bounded by $\mu_A(x_1) = \mu_A(x_2) = \mu_A(x_3)$, $\mu_A(x_1) = \mu_A(x_2) = \mu_A(x_4)$ and $A_{\frac{1}{4}}$. Call this surface $\varpi^A_{x_1x_2}$. We can define $\varpi^P_{x_ix_k}$ in a similar manner.

Define $\varpi^P_{x_ix_j}$ to be the surface $\mu_P(x_i) = \mu_P(x_j)$ bounded by $\mu_P(x_i) = \mu_P(x_j) = \mu_P(x_i) = \mu_P(x_k)$ and $\mu_P(x_i) = \mu_P(x_j) = \mu_P(x_l)$ where $i \neq j \neq k \neq l$ and P is an arbitrary player.

Now draw $\varpi^A_{x_1x_3}$, $\varpi^A_{x_1x_4}$, $\varpi^A_{x_2x_4}$, $\varpi^A_{x_3x_4}$, and $\varpi^A_{x_2x_3}$. This divides the 3-simplex into 4 regions. Label them A_i, where x_i is the vertex that bounds that particular region. Label the cut-sets in each region by the label of the region, and label cut-sets by the name of that region. Label cut-sets on the bounding surfaces by the regions they border.

Draw the rest of the 18 surfaces $\varpi^P_{x_ix_j}$, cycling i and j through (1, 2, 3, 4) and cycling P through (A, B, C, D). For a certain P, label the regions into which the six surfaces associated with that P divide the 3-simplex in a similar manner as described for player A.

The 24 total surfaces divide the cake into a number of regions. Again, there is a 3-D region, call it Ψ, labeled A_i, B_j, C_k, D_l where $i \neq j \neq k \neq l$. It is also bounded only by surfaces, and cannot have an edge or face as a bound.

The rest of the proof that our algorithm gives an envy-free division is similar to the 3 person case. Choose a player who prefers an endpiece in region Ψ. Without loss of generality, assume that player is player D and that he prefers piece x_4. Note that for a fixed x_4, the solution space of possible cut-sets is a 2-simplex. The justification for this is simple and

similar to the three person case.

We have already proved that on the 2-simplex there will be an envy-free region[5] Ξ[6]. Much as we proved the existence of ι (the envy-free cut-set generated by the algorithm) by showing that Ξ intersected λ and shared a boundary where a cut-set in both regions could be found, we'll prove that the algorithm generates a envy-free cut-set for four players by showing Ψ intersects Ξ and shares a boundary.

Note that the region Ξ in the 2-simplex will be bounded by at least three of the nine curves $\mu_A(x_1) = \mu_A(x_2)$, $\mu_A(x_1) = \mu_A(x_3)$, $\mu_A(x_2) = \mu_A(x_3)$, $\mu_B(x_1) = \mu_B(x_2)$, $\mu_B(x_1) = \mu_B(x_3)$, $\mu_B(x_2) = \mu_B(x_3)$, $\mu_C(x_1) = \mu_C(x_2)$, $\mu_C(x_1) = \mu_C(x_3)$ and $\mu_C(x_2) = \mu_C(x_3)$. A cut-set will therefore be chosen that is on at least one of these bounding curves. However, in the 3-simplex we see that at least one of those same surfaces help to bound region Ψ (see appendix **A.4**). Moreover, for a cut-set to be in Ξ is a weaker condition than for it to be in Ψ, so every point in Ψ will also be in Ξ. Therefore, all we need is for a cut-set to be chosen on a bounding curve of both Ξ. We see that because Ψ has that same bounding curve of Ξ and $\Psi \in \Xi$, there will be some cut-set ν on the curve that is in both Ξ and Ψ. Using the three player version of Lemma 2, we see that since a cut-set on that curve can be generated by our algorithm for some permutation of players [7], then ν can be generated as well. Since $\nu \in \Psi$, the algorithm gave an envy-free cut-set for a certain permutation of players.

Another way to think of this is that beginning at $x_4 = 0$, we gradually increase x_4, "shifting" the 2-simplex toward the vertex (0,0,0,1). At every fixed value of x_4, there will be a unique cut-set on the resultant 2-simplex for an ordering of A, B, and C. Therefore, as the 2-simplex "shifts", the cut-sets will trace out a continuous curve (this is because of the continuity of valuations, one of our basic cake-cutting assumptions). The envy-free cut-set for four players is the intersection of this continuous curve with Ψ. There will always be an intersection by the previous paragraph.

Note that we can generalize Lemma 2 to four players now.

[5]Where everyone is envy-free

[6]Much as we generalized the definition for λ to the 2-simplex, generalize Ξ to mean the region where all cut-sets are labeled A_i, B_j and C_k, where $i \neq j \neq k$

[7]This is simply an application of the proof that we can generate an envy-free cut-set for three players on any boundary curve except for $x_1 = x_3$. However, appendix **A.4** discusses we can always choose another boundary curve besides $x_1 = x_3$

Also note that by changing the order of who offers, we could find a cut-sets on different boundary surfaces, as long as that boundary surface doesn't have the form $\varpi^P_{x_1 x_4}$ (see appendix **A.6**).

The extension of this to n-players is straightforward, and follows the same principles as for the four and three person case, with Lemma 2 generalizing every time to n-players.

6 Other rules for the algorithm

The rule that a player must go to the end of the order when he rejects a piece is required to cover a case not mentioned in the preceding argument. Although the previous proof demonstrates that a certain permutation of players following the original specified ordering and "greedy" maximin preferences (that is, they do not strategically think ahead to try to get the largest piece) get an envy-free solution, a player may be able to manipulate the algorithm to get more, even if the proposed division was envy-free for him anyway. This rule eliminates that possibility.

Proof: Let the player being offered to be player D, and the piece being offered be x_1 without loss of generality. Since if D rejects the piece he will *always* be the one being offered to (he is now the last player in the ordering), he will never get a larger piece than the first offer. That is because in the algorithm, it is the players who do the offering that choose where to cut. To minimize the size of the piece being offered (and thus to maximize the cake left over for their own piece), the offerers will always cut near the boundary curve or surface (ie. $\mu_D(x_1) = \mu_D(x_2)$) of player D and offer player D a barely acceptable piece there.

One way to think of this is that after D accepts an offer, the other players divide the rest of the cake envy-freely amongst themselves (at a cut-set η) and ignore player D's valuation. Therefore, it is very probable that there may result a piece, say x_2 that D values highly. The algorithm stipulates the creation of another piece x_1 that ties in value with the x_2. However, if D rejects the piece, at best the other players will cut another piece for D tied in value with x_1. Consider cut-set η. D will prefer one piece created by that cut-set (x_2). If this piece is not preferred by any other player, then we are done, and D will not have received a larger

piece.

Otherwise, if another player also prefers x_2 the only way for D alone to prefer that piece is if we choose another cut-set (envy-free for the other players too), say κ, that *reduces* the size of x_2 to such a degree that either D or the other player prefers another piece. If the other player prefers a different piece, then D will have gotten a smaller portion of cake than x_1 (the reduced x_2). Otherwise, D will get the different piece, say x_3. However, x_3 cannot be bigger than x_1, because in the new cut-set κ, the reduced x_2 is the second most valuable piece according to D. Therefore x_3 needs to be only slightly more valuable to appeal to D (in practice, it can be considered equal in size to reduced x_2). □

If the rule was not included, then it would be possible for D to get more of the cake by rejecting x_1, because for x_3 to be equal to reduced x_2 requires another player to cut the cake for D (see appendix **A.1**).

7 Applications and Refinements for the Algorithm

The assumption of complete information in cake-cutting is not too far removed from the "real world". Often the players needing mediation in dividing goods already have a long-standing conflict or relationship, as is the case in divorce proceedings or in international conflicts. Thus, they are likely to know each other well enough to assess each other's desires. Sometimes the assumption of complete information is more realistic than that of incomplete information and also leads to more resilient algorithms: although incomplete information cake-cutting algorithms can be manipulated with complete information, or "inside" information, our algorithm cannot be.

How could our algorithm be applied to conflicts or disputes? The most important concern is the ordering of the players involved, as only one permutation is guaranteed to work. However, the players who prefer endpieces will sometimes have an incentive not to offer, but be offered to, so that could be a starting point for creating an ordering; the players who prefer endpieces declare themselves (see appendix **A.6**). Other possibilities for creating an ordering include letting an impartial referee recommend an ordering, or simply trying every

permutation until one works (of course we could not actually destroy the good if we use this method).

Once we do have an ordering, however, our procedure is simple to implement and follow, unlike other general information procedures, which require an unbounded number of cuts and are up to three pages long. A more refined version of the algorithm (see the below section) could be used to resolve many conflicts over goods, an especially important goal in this turbulent age.

8 Future Directions

Although we have presented a general n-person algorithm for complete information situations, the algorithm only gives an envy-free cut-set with a certain permutation of players (at least one permutation works). Finding a general algorithm that works for all permutations of players would be a logical next step. Even if such an algorithm may require more than $n-1$ cuts, it may still prove to be more useful in practical situations if the number of cuts involved is not too high.

Of course, the question of whether or not there is a bounded general algorithm *without* complete information is still open.

9 Conclusion

In this paper, we introduced a new complete information general person envy-free minimal cut procedure for dividing a cake, which is certainly extremely useful. We see that complete information indeed helps in terms of creating bounded algorithms and that is certainly worth pursuing. However, in some other ways, complete information algorithms are harder to analyze because it is necessary to take into account manipulations by players which were not possible before. Still, in the future, the assumption of complete information could lead to simpler and more practical n-player bounded cake-cutting schemes. Most importantly, we have demonstrated the viability and usefulness of complete information in cake-cutting

through the creation of an algorithm.

10 Acknowledgments

I would like to thank my mentors, Professor Brams and Professor Greenleaf at NYU, and Mr. Joungkeun Lim at MIT, for their invaluable help and guidance throughout this project. I would also like to thank CEE for giving me the opportunity to pursue this research over the summer at RSI.

References

[1] J. B. Barbanel and S. J. Brams. Cake Division with Minimal Cuts: Envy-Free Procedures for 3 Person, 4 Persons, and Beyond. *Mathematical Social Sciences 48* (2004), 251–269.

[2] S. J. Brams and A. D. Taylor. An envy-free cake division protocol. *American Mathematical Monthly 102* (1995), 9–18.

[3] S. J. Brams, A. D. Taylor, and W. S. Zwicker. A Moving Knife Solution to the Four-Person Envy-Free Cake-Division Problem. *Proceedings of the American Mathematical Society* (1997), 547–554.

[4] L. Dubins and E. Spanier. How to cut a cake fairly. *American Mathematical Monthly 68* (1961), 1–17.

[5] T. Ichiishi and A. Idzik. Equitable allocation of divisible goods. *Journal of Mathematical Economics 32* (1999), 389–400.

[6] J. Robertson and W. Webb. **Cake-Cutting Algorithms.** A K Peters, Wellesley, MA (1998).

[7] F. E. Su. Rental harmony: Sperner's lemma in fair division. *American Mathematical Monthly 106* (1999), 930–942.

A Appendix

A.1

Another player must always be cutting for (and offering to) D because that is the only way to minimize the envy-free piece that D will receive, as mentioned earlier in section 6. If D

is doing the offering, he will minimize the size of the piece offered (and thereby maximize the value of his own piece later), rendering our argument invalid because we can no longer be certain x_3 will be equal to reduced x_2.

A.2

Player A offers to B after player B offers to C because of the ordering $\underline{ABC}$. Because A maximizes his portion of the cake by minimizing player B's portion, A will cut near the bisection point of player B's valuation function. In the case of a fixed x_3, as in the example, the bisection point of B's valuation function is where $\mu_B(x_1) = \mu_B(x_2)$, or simply $\varphi^B_{x_1 x_2}$.

A.3

We can see that it would be an $n-1$ simplex by considering the equation $x_1+x_2+\ldots+x_n = 1$ in $\Re^n$, and then taking the intersection of that with the first 2^n-tant.

A.4

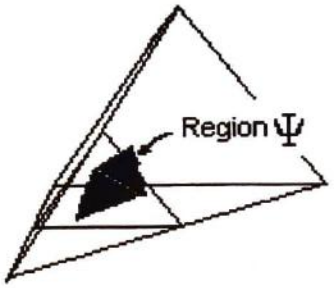
Notice that the bounding curves in the $n-1$-simplex have the same equations as the surfaces in the n-simplex. Now consider the *envy-free region* [8] in the n-simplex (call it Γ). It is always bounded by the curves of the form $\mu_P(x_i) = \mu_P(x_j)$, with every possible value for i and P appearing at least once, although for each i there will be only one P.

The region of the n simplex (projected onto an $n-1$ simplex) is always contained in the envy-free region of the $n-1$ simplex (call this region Σ). Moreover, sufficiently close to region Γ the region Σ shares at least some boundary curves with Γ.

This fact follows from the pieces the players prefer in each region. Without loss of generality assume that at Γ, player A prefers piece x_1, player B prefers x_2 and so on. Thus the boundary curves will be $\mu_A(x_1) = \mu_A(x_i)$, $\mu_B(x_2) = \mu_B(x_j)$, $\mu_C(x_3) = \mu_C(x_k)$, etc, with $i, j, k \ldots \in (1, 2, 3 \ldots, n)$. There be an unique region Σ in the $n-1$ simplex where the first $n-1$ players prefer the same pieces that they preferred in Γ. We can think of this by taking

[8] We define envy-free region to be a region where the cut-sets are labeled $A_i, B_j, C_k, \ldots$ where $i \neq j \neq k \neq \ldots$. In other words, it is the generalized version of the regions λ and Ξ discussed earlier

the intersection of a $n-1$-simplex with Γ (a 2-simplex example is given below, with the shared boundaries highlighted in red). We can only remove at most one constraint [9] (boundary) on the $n-1$-simplex, and still satisfy the requirement that in Σ the first $n-1$ players prefer the same piece that they preferred in Γ. However, if only one constraint (boundary) was removed in "switching" from region Γ in the $n-1$-simplex to region Σ, then there will be at least $n-2$ constraints (boundaries) that will be the same in Σ and Γ.



In the following paragraph x_i and x_k are endpieces in the $n-1$ player game and the following argument itself addresses the issues raised by **A.7**. Since the boundary that was removed can also be part of a mostly similar $n-2$ constraints for a different permutation of players there are $n-1$ possible boundary curves (the $n-2$ constraints plus the boundary that was removed). Since at most two boundary curves can have the equations $\mu_P(x_i) = \mu_P(x_k)$ and there are $n-1$ possible boundary curves, by the pigeonhole principle there are at least $n-3$ surfaces that both do not have the equation $\mu_P(x_i) = \mu_P(x_k)$ and are shared between Γ and Σ. We see that therefore there will always be a cut-set on a boundary curve that will be both in Γ and Σ.

A.5

The existence of the region mentioned is another way of stating that there exists some envy-free partition of the cake using only $n-1$ cuts. The only way to get an envy-free partition using only $n-1$ cuts is if each player prefers a different piece. We model that with our

[9] This constraint would relate to player n, who prefers x_n. Since we are considering the $n-1$-simplex, we can ignore player n, piece x_n, and that boundary curve.

labeling. For example, at any cut-set labeled A_1, player A prefers piece x_1. If there is in fact no such region, that must mean that two players must prefer the same piece, which would contradict the existence proof of envy-free partitions.

A.6 An Observation on Endpieces and Players

Sometimes, it is impossible for players who prefer endpieces to offer a piece to another player without becoming envious themselves later on. This happens when the curve that bounds the envy-free region (call the region Ω) is $\mu_P(x_1) = \mu_P(x_n)$, where n is the number of players and P is the player in question. This also has to be the only such curve of the form $\mu_P(x_i) = \mu_P(x_j)$ for P to be unable to offer a piece without becoming envious later on.

It is easy to see why. Without loss of generality assume P offers x_1, one of the endpieces. Because P is not getting x_1, he will have to get piece x_n to be envy-free. However, this is never guaranteed because even in the three-player case since other players may never cut at $\mu_P(x_1) = \mu_P(x_n)$. After all, at a fixed x_1, the other player will cut at $\mu_P(x_2) = \mu_P(x_3)$, which does not have to intersect Ω.

Players who do not prefer endpieces do not suffer from this problem, because what causes the problem for P is that he must offer an endpiece, which he values equally at $\mu_P(x_1) = \mu_P(x_n)$. It is not possible to get an envy-free cut-set if we offer a piece that is part of the equation of the sole bounding curve. That is, if only $\mu_P(x_i) = \mu_P(x_j)$ bounds Ω, P cannot offer either i or j and get an envy-free piece [10]. This is not a problem if i and j are both not endpieces, as P can offer k a different piece (also k is an endpiece) instead. However, if both are endpieces, then P can only offer i or j. Thus P will have to offer too big a piece to get an envy-free piece later on (ie. the final cut-set will not be in Ω).

[10] That follows from the fact that clearly other players who offer to P will only make an offer at the boundary curve $\mu_P(x_i) = \mu_P(x_k)$ (assuming P offers j) where $k \neq i \neq j$. But if $\mu_P(x_i) = \mu_P(x_j)$ was the only curve of that form bounding Ω, $\mu_P(x_i) = \mu_P(x_k)$ obviously cannot bound Ω by definition, and thus no cut-set on that curve is part of Ω.

A Practical Implementation of Gradient-based Convolutional Neural Networks in Handwriting Recognition

Yuetian (Peak) Xu

under the direction of
Yann LeCun and Davi Geiger
New York University Computer Science Department
and Center for Advanced Technology

Stuyvesant High School
November 15, 2004

1 Introduction

Recent improvements to both the power and the cost-effectiveness of computing bring the dream of automation closer to reality. In particular, implementations of systems such as real-time speech and handwriting recognition are now quite common thanks to progress in the fields of pattern recognition and machine learning. This paper offers some practical techniques for improving upon the robustness and accuracy of certain existing handwriting recognition systems.

A number of different improvements on such systems are currently under intensive research. Among these are *Support Vector Machines*, *Bayesian networks*, and variational learning. Despite a recent lack of interest, several computer scientists believe that neural networks will turn out to be one of the most effective techniques for handwriting recognition. In particular, studies by [1] show convolutional variety to be likely the best technique for this task.

A challenge for many recognition strategies is that machine learning techniques tend to be based on hand-tailored heuristics. [2] have demonstrated that it is possible to replace these heuristics with learning machines that can operate directly with pixel images and still realize gains in accuracy. They also implemented their idea in the form of a convolutional neural network known as *LeNet 5*. LeNet 5 was used to recognize numerical digits, specifically for the purpose of postal code recognition. They were able to attain previously unachievable accuracy with LeNet 5, as measured with the benchmark MNIST database for handwriting recognition [3].

LeNet uses a relatively small number of character classes (10), and may not have demonstrated the full power of the technique. This generated considerable interest in the problem of extending LeNet 5 into a full fledged handwriting recognition system. The main portion of this paper will discuss new techniques for increasing the number of character classes discernible by LeNet 5. The admissible input is extended to the entire printable ASCII set, taking samples from the Unipen database [4]. The increased number of classes creates some persistent complications, partially resolved by our techniques. The eventual goal is to use an unrestricted character set, with applications including the character sets of other languages, as well as custom symbol sets. The improved system will soon be integrated into a commercial handwriting recognition product and will serve as the ultimate practical test of the implementation.

2 Background

2.1 Why not SVM?

Support Vector Machines (SVMs) have received a great deal of attention recently due to their versatility. However, SVMs are not at their best when dealing with character recognition [1]. To understand this, it may help to examine the theory of SVMs. SVMs classify vectors in two categories using a *classification function* of the form:

$$g(X) = \sum_i \alpha_i K(X, X^i)$$

where X is the vector to be classified, X^i is the i-th training sample, $K(X, X^i)$ is a so-called *kernel function* that measures the dissimilarity between X and X^i, and the α_i are coefficients computed by the learning algorithm. When $g(X)$ is larger than a certain threshold, X is assigned to class 1, and when $g(X)$ is less than the threshold, X is assigned to class 2. The SVM learning procedure minimizes a loss function whose solutions tend to have only a few α_i that are non-zero. The X^i that correspond to a given non-zero α_i are called the *support vectors* of that α_i.

This greatly simplifies calculations from a theoretical point of view by using linear separations (albeit in high dimensional space), relative to so-called polynomial classification methods. It has also been proven to produce the optimal result in a general learning task.

However, it is not very well suited to handwriting recognition, since the higher-dimensional spaces in which linear separation are generated by taking Cartesian products of real spaces. This means that there must be fairly exact feature values. But exactness is notoriously difficult to achieve with handwriting. Also, it is difficult to conceptually define the axes of the higher dimensional spaces. If the feature space referred to had axes based on curvature and slope, for instance, there would be a problem in interpreting the product of these axes.

More importantly, SVMs are unsuitable because they use "global template matching". Each kernel function $K(X, X^i)$ can be viewed as a measure of similarity between the pattern to be recognized and one of the training samples. Performing global comparison (template matching) is not effective when the patterns have many variabilities (size, position, orientation, stroke width, slant, writing

style, etc.), because a vast number of templates would be needed to span the space of all possible variations. Systems that learn local features (like convolutional networks) are better at handling such variations.

In view of this difficulty, techniques such as neural networks seem more promising. Furthermore, it has been proven that traditional neural networks, using *boosting*, are able to attain the accuracy of SVMs asymptotically [5]. Thus, even from a theoretical perspective, neural networks result in no accuracy penalty, provided boosting is used.

2.2 Introducing Gradient Descent

In learning theory, there are several learning strategies. One of these is the *Gradient Descent Rule* used by LeNet. The version of the gradient descent rule used in LeNet 5 is in fact an evolution of the *Widrow-Hoff Rule* or the *Delta Rule*. An important idea of Widrow and Hoff was labeling outputs with -1, 1 instead of 0, 1. (For details, see [6], [7], and [8].)

The gradient descent method prompts the machine to minimize an error or energy function such as the following:

$$E = \frac{1}{2}\sum_{m=1}^{M}(y^m - t^m)^2$$

where t is the target, y is the output, and E is the total error over all sets of weights. Minimizing the difference between the target and output iteratively and updating the weights minimizes the energy function E. It is also desirable to achieve the minimization in as few iterations as possible. For this reason, it is often assumed that the energy function is quadratic, or at least locally quadratic. For example, moving from 0 to the minimum of the function $(w-3)^2$ can be done in one step. Just take the derivative of the function at 0, which is the gradient in this context. This result of $2w - 6$ yields a value of -6 when $w = 0$. Evaluation of the second derivative yields a value of 2, and inverting yields $\frac{1}{2}$. Multiplying the first derivative by the inverse of the second yields -3, the signed distance to 0 from the minimum of the function (which occurs at 3)(see Figure 1).

This method can be extended to any quadratic error surface by taking the gradient vector along the vector to the minimum point and multiplying it by the inverse of the *Hessian matrix*. The Hessian is the generalization of the second derivative in higher dimensional space. The inverse of the Hessian, an $n \times n$ matrix in n dimensions, is used to compute the optimal learning rate for

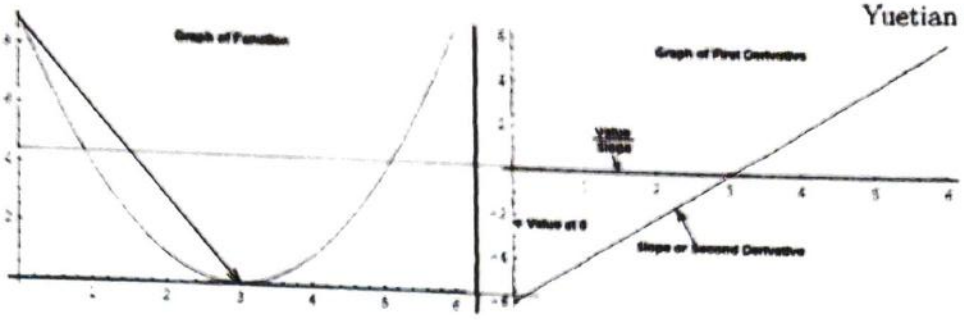


Figure 1: The Gradient Descent Rule

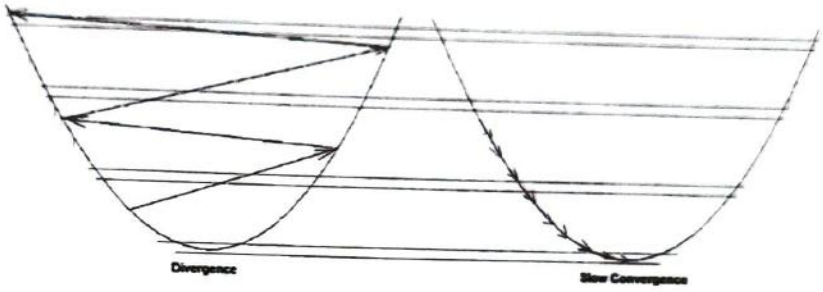


Figure 2: Problems with incorrect learning rates

convergence in a single step.

This brings up a performance consideration. The computation of the inverse of an $n \times n$ matrix requires $O(n^3)$ time, and is very expensive for large n. For this reason, LeNet-5 uses only the diagonal entries of the Hessian and approximates the inverse in $O(n)$ time. This approximation is sufficiently accurate to prevent the twin evils of divergence and slow convergence that occur with too large and too small learning rates respectively. (See Figure 2)

2.3 Introducing Convolutions

Convolutional neural networks were taken as the basis of LeNet 5 and other systems because of their unique ability to account for the specific properties of the input.

Fully connected networks, by their nature, cannot distinguish between permutations of pixels. They would thus result in undesired spatial ambiguities when applied to handwriting recognition. A reduced number of connections and increased efficiency makes convolutional networks preferable to fully connected ones.

Other neural network types such as *Time Delayed Neural Networks* (TDNNs) are certainly worthy of attention. However, these often rely on temporal data, such stroke order etc. that is often

unavailable. Furthermore, the fact that they work with temporal data implies that "topological" features or geometrical features that are close spatially but far apart in time will not be captured. Finally, the human visual system has the highest performance of any offline character recognition system built today and does not require any temporal information.

3 System Overview

3.1 Lush

The LeNet 5 system is programmed in Lisp Universal SHell (Lush), a language that has lisp syntax and can be run in interpreted mode but can also be compiled into C object code where the dynamic loader of the Lush runtime is able to load and execute (see Appendix B for sample code). This system has considerable flexibility and also allows inline mixing of C code, making it easy to develop.

3.2 Character Recognition System

Many complications arise from the increased number of classes. In addition, the nature of handwritten characters for the entire printable ASCII set is quite different from that of the numerical digits.

In LeNet 5, there is no distinction between printed and cursive letters. This makes the classes far harder to define precisely. Some letters, such as "Q", admit significant difference between the handwritten and printed versions (see Figure 3).

Figure 3: A cursive 'Q' and a printed 'Q'

Due to these and other technical challenges, initial testing revealed an accuracy rate of approximately 55%, and a roughly 48% longer training time. In order to improve both accuracy and speed, the system was examined as a whole and each part underwent various revisions. The revisions were constrained by the need to integrate the parts into a single working product. Figure 4 shows the basic structure of LeNet 5.

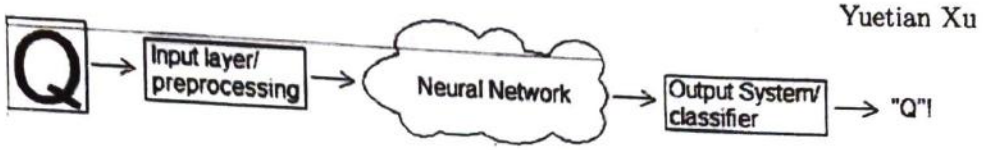


Figure 4: LeNet 5 Structure

In front is the preprocessor and data preparation module. The nature of the input and the kind of processing it undergoes can be varied. In the middle the neural network, which uses hidden units and feature units to learn by extracting features and patterns impossible for a human. Despite the "mystique", the size of the cloud and the rules by which the network learns can also be changed. At the end there is the classifier module. Potential adjustments here may affect such elements as the classification metric, the back-propagation updating, and the size of the validation database.

4 Data Preparation and Preprocessing

4.1 The Data

The input data were largely taken from the Unipen database. The entire printable ASCII set was included. The alphabetic characters were taken in isolated form from the Unipen database; numerical digits were taken from the MNIST database. The non-alphanumeric characters were extracted from sentences and phrases of the Unipen dataset.

The Unipen dataset included the information in a XML-like tagged format. Some keywords, such as "PEN_UP" and "PEN_DOWN", provide information such as pen state and the identity of writers. There are also coordinate pairs that determine the position of the pen. Such precursors of the input must be extracted and processed first.

4.2 Creating Images

LeNet 5 accepts a standard 28×28 pixel input. Also, for best results, the characters are drawn so that lines are 1 pixel wide. The original Unipen data had widely varying ranges. Thus, the first task was to connect the coordinates and also scale them down to the proper size for LeNet 5. The GD image library for C based upon the Bresenham Line Drawing Algorithm was used to generate the images.

The coordinates were first scaled down to the 28×28 grid and then connected with lines; this avoided discontinuities which would occur had the process been reversed. The images were sorted

into various directories according to the class that they represented.

4.3 Multiple Input Planes

Networks learn from the input. Thus, learning is sensitive to forms of representation of the input. Time Delayed Neural Networks have an additional time parameter to help with the recognition. More parameters can be produced for the network if instead of images, multiple layered inputs are used. This was generated by preprocessing on the original Unipen data, via slope fields. So each layer represent the response of a particular direction/slope to the image. There was no weight system to make the data in one layer count more than any other. The planes must represent equally weighted parameters. Slopes in different directions were chosen as the parameters.

Using a look-ahead algorithm for contour tracing, local slopes were calculated for every black pixel on the image. Then, a set of 4 basis vectors was used for the expression of the slope. Actually, since the image is in two-dimensional space, two non-parallel bases should be sufficient for the purposes of defining the slope. Unfortunately, two bases are often insufficient to establish meaningful correlations as the projection on one may be all clumped together (see Figure 5). Two additional bases at 45° to the existing ones turned out sufficient to correct the problem.

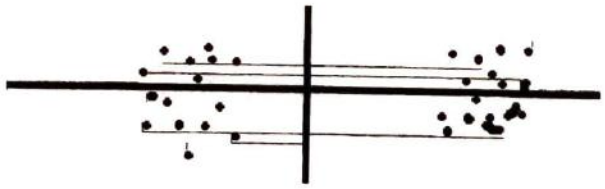


Figure 5: Projections on the axes insufficient for disambiguation

A slope field was created from the image and then expressed in terms of each basis by having the corresponding point on the corresponding layer "filled" with $\cos^2(\theta_s - \theta_r)$ where θ_s is the slope at the point in terms of angle from the positive x-axis and θ_r is the angle of the basis. The four bases give rise to a four layer input system. Originally, the values were of float type. Due to RAM restrictions, scaling down of floats into unsigned bytes made for little loss in accuracy and a quarter of the space.

The results are presented in Table 1[1]. Use of four input planes instead of one yielded significant

[1]The top n accuracy counts a classification as correct if the correct output is within the top n when ranked by score. This definition is used throughout this paper when discussing results

# in Test Set	# of Planes	Accuracy	Top 3	Top 5	Avg. Running Time (s)
2,000	4	88.70%	93.70%	N/A	1159.03
2,000	1	84.90%	91.50%	N/A	405.16
8,000	4	85.15%	91.15%	93.26%	1186.62
8,000	1	85.12%	91.19%	93.33%	864.96
10,000	4	85.25%	91.66%	93.50%	1245.62
10,000	1	85.76%	92.36%	93.98%	850.91

Table 1: Input Comparison—Extra planes no justifiable improvement in long run

gains in smaller test sample cases but losses in larger cases. This can be explained by the fact that the slope fields were generated from the same data as the images and hence no independent information was added. While this helps the network when fewer samples are available, it hinders the system when the samples are more numerous, since the network is capable of finer adjustments. The first few iterations of the four plane version had higher accuracy, but was soon overtaken by the one plane version as more training occurred.

The doubling of the running time of the one plane version from 2,000 test samples to 8,000 test samples is due to need to access swap instead of physical RAM. LeNet uses both the testing and training sets constantly. Thus when the number of test samples increased, the combined total was greater than the physical RAM and hence there was a need to access the hard drive for the swap file.

4.4 Packing the Images

The images were then packed into separate image and label files for LeNet 5 using Lush matrices with one set each for training and testing. The sets contain 50,000 and 25,000 samples, respectively. Of the latter, a varying number was used for different testing runs.

Care was taken that the training data and the testing data were drawn from different writers and thus there was no correlation between the training data and testing data. This ensures that the accuracy from the system is not artificially inflated.

Some topologically indistinguishable characters such as "O", "o", and "0" were merged together into the same class. Hence, instead of the 95 classes of the printable ASCII set, 80 classes were created. This is harmless since context will be sufficient for disambiguation in the commercial product (see Section 7.2). This renders exact identification unnecessary, and enable use of confidence intervals

instead.

4.5 Expansion of Dataset through Transforms

Some affine transforms, such as translations and dilations, were used to create new versions of the data to compensate for the paucity of available training data. It is known that the more data that the neural network has to work with, the better it usually performs. In fact, the increase of training data by up to 3 orders of magnitude can result in dramatic increases in the accuracy rate [9]. Inflation through transforms was particularly necessary. In some cases, there were only 200-300 samples per class while in MNIST, there were 6,000 per class.

The transformations are done in memory for the following reasons. First, the pre-expanded set requires a great deal of reading and writing to the hard drive, making it far less efficient than manipulation in RAM. Secondly, the size of the database used for training is limited by RAM, as is accuracy, if the training is split over multiple sessions. Thus, transforms built into the program enable more training data to be used.

In a test, the data source (input/preprocessor for network) module for LeNet 5 was changed to create bilinear transforms. For this initial test, rotation, scaling, and translation were included. 41 combinations of rotation by 5° increments ranging from $-15°$ to 10°, 15°, scaling by a 80%, 90%, 100%, or 105%, and translations by 2 pixel increments from -5 to 5 pixels were used as templates (precomputed sets of transforms).

The use of templates reduced the computation necessary to perform these bilinear transformations. The theory of bilinear transformation rests on the fact that a mapping of the lattice points of an input plane to an output plane using a given function may not alway be one-to-one. Given that the function takes a real value at each lattice point of the input plane, the most accurate mapping to the output plane is desired.

Using an inverse mapping, each lattice point on the output plane is mapped to a point with real coordinates on the input plane. This point will be surrounded by a square of four lattice points (except for the degenerate case of actually being on a lattice point in which case the solution is trivial). First horizontal coordinates are considered, and two pairs of points with equal vertical coordinates are given weights based upon their relative horizontal distances from the point. Then,

weights are given according to vertical distances. Each of the four lattice points now has two weights which are multiplied together to produce a total weight for that point. The real value at each of the four lattice points is then multiplied by the corresponding weight and then summed to produce a certain real value for assignment to the lattice on the output plane. (See Figure 6)

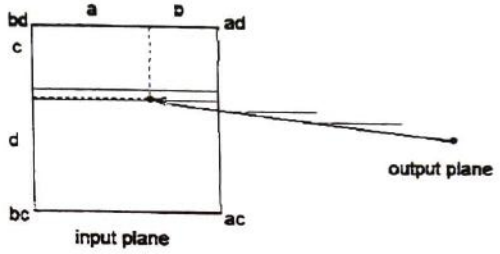


Figure 6: Bilinear Transformation Mapping showing distances and corresponding weights at lattices

This procedure was implemented in C for each of the 41 combinations. Each input had some templates applied to expand the dataset. The reason for the 41 templates instead of the total possible 168 is that some combinations are unnecessary. The use of a fraction that is strategically selected to go across all categories is already sufficient to teach the machine all the necessary invariances. Similarly, not all 41 combinations needed to be applied to each sample, and a subset of the 41 was applied to each sample with that subset cycling through all 41. The size of the subset is varied for different runs. The choice of 41 in this case is due to its primality, making it relatively prime to all the numbers smaller than it. Hence, one is free to cycle using any number of templates per sample and be assured of a minimum number of samples having the exact same subset of templates applied.

Preliminary testing with seven templates applied to each sample showed that the number of iterations necessary to obtain comparable performance to previous machines is less than a tenth of what was previously required. However, the time per iteration was quadrupled. Thus, reaching comparable performance took roughly 35-40% of the original time. Further testing is necessary to quantify the exact amount of improvement in accuracy.

4.6 Blur/Elastic Distortions

The affine transforms may not be sufficient to teach all the character invariances. To add some topological invariance, blur and elastic distortions were applied to the input. In order to do this, a

vector was generated at each point on the image of random length (up to 3 pixels) and direction. The vector field applied thus produces a new image. Then, the value of each pixel on the new image was changed to the average value of its eight neighbors and this produced the necessary blur to smooth the image.(See Figure 7) This distortion was applied several times dynamically during training to promote invariance, and resulted in a performance improvement similar to the affine template transforms.

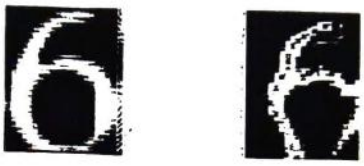


Figure 7: Elastic distortions on '6'

5 The Network

5.1 Scaling

It is known that the size of the neural network (essential the number of feature units) can have a significant impact on the performance [1]. In [1], it was claimed that for the MNIST database in which ten classes are used, fifty feature units would be ideal for the second layer of the network. From this, it appears that this system, which has 80 classes, might very well need hundreds of units compared to LeNet's default 16 units.

The most easily variable number of feature units occurs in the second layer, and numbers 16 in LeNet 5. There is a set of 6 feature units in the first layer. Thus the maximum number of connections that one can use without redundancy is $2^6 - 1 = 63$ units.

Hence, a 63 feature unit second layer was constructed and tested. The results are seen in Table 2. The addition of extra feature units provides a noticeable performance boost in all accuracy categories, but the increased size of the network almost triples the running time.

# in Test Set	# of Feat. U's	Accuracy	Top 3	Top 5	Avg. Running Time (s)
8,000	63	86.16%	91.60%	93.71%	2536.18
8,000	16	85.12%	91.19%	93.33%	864.96

Table 2: Feature Unit Comparison—More is better

6 Output Modules

6.1 Classifier

The classifier module is the module that calculates the scores for each of the classes and then decides which class(es) should be considered match(es) for a given input.

The LeNet 5 system employs a "max-classer" for this task, and selects the class that contains the greatest value. This is based upon a distance calculation for the closest class. Related systems use of a *feature table* consisting of an 80×80 identity matrix.

However, a table of features, together with a weighted Euclidean distance classifier, may be able to describe the characters equally well. A "wedist-classer" was coded for this purpose. This calculates the weighted Euclidean distance between points in the feature space. Weights are necessary, since some features account more for the discernment of characters than others.

6.2 Feature Table

A feature table was developed according to the geometry of the characters. Some examples include "ascenders", "descenders", "loops" and also colorful names such as "kickers" in order to describe various aspects of the ASCII character set. The feature table contains 32 features and appears sufficient to describe the entire ASCII dataset. (See Appendix A for portions of the table.)

Experimentation with 2,000 samples demonstrates that "wedist" results in negligible decrease in accuracy, but uses only half the training time. In the 10,000 and 8,000 test sample cases, in lieu of a significant decrease in training time, the accuracy is increased for the top class. In all cases, it appears that max-classer is roughly equal or inferior to the wedist-classer when computing the accuracy for the top class. However, it is marginally better at getting the correct class into the top 3 or top 5 spot.

# in Test Set	Classifier	Accuracy	Top 3	Top 5	Avg. Running Time (s)
2,000	wedist	84.90%	91.30%	N/A	405.16
2,000	max	85.00%	94.60%	N/A	872.56
8,000	wedist	85.12%	91.19%	93.33%	864.96
8,000	max	82.00%	91.21%	*93.54%	887.57
10,000	wedist	85.76%	92.36%	93.98%	850.91
10,000	max	83.21%	92.51%	94.56%	906.68

Table 3: Classifier Comparison—wedist is better

6.3 Transforms/Expansion for Testing

Expansion of the training set through transforms improved the accuracy of the system. Similarly, use of transforms on the testing set can also produce improvements. Instead of directly outputting the results of the test image, the input image can be distorted/transformed into several different versions and each can be fed into the machine. The scores of all the outputs can be combined to yield a total score. This can provide a more accurate and stable result and is almost like boosting. (See Section 6.4) The different iterations in which the transformed testing sample is fed through the network can be though of as the machines in boosting which collectively make a decision, forming a larger machine. The result of three distortions per testing sample is an increase of approximately 170% in running time with a gain of 3-5% in accuracy for the top class. The accuracy gain is less for the top 3 and top 5.

6.4 Boosting

Boosting is a concept often used to improve the results of a network by training another copy of the network on the samples for which the first copy erred, and then yet another to compensate for the mistakes of the first two, etc. In principle, boosting yields arbitrarily high accuracy.

This concept is used mostly for simple classifiers such as linear classifiers and this can make such 'weak' classifiers become jointly very powerful. One often sets a target accuracy rate and add extra machines until that target is reached. One would also have a weighted sum of the scores of the machines to produce the final score of the boosted system. This type of *additive boosting* is used on simpler machines mainly due to the fact it is very computationally expensive to maintain many copies. It is not possible to do this with the entire LeNet 5 system, as the training time would be prohibitive. In preliminary experimentation, the last layer of LeNet was removed and then the output of the second to last layer was treated as the input of an additive boosted machine, trained to classify from the outputs of the second to last layer. This led to no significant improvement in performance, and it appears that the output of the system was essentially determined by the second-to-last layer.

The type of boosting finally implemented was of a simpler variety. A machine was trained as usual. Then, the second was trained only on the error cases of the first. A third machine was trained only

on the samples on which the first two machines disagreed. Then each of the machines produced as an output a class and a score for that class. Since the first machine presented with the most input data[2] and hence most training, it should be given the greatest weight in most cases. Only when the second and third machines agree, should the decision of the first be overridden. This scheme turned out to be both effective and parsimonious. The results are presented in Table 4.

# in Test Set	Boosted	Accuracy	Top 3	Top 5	Avg. Running Time (s)
8,000	Yes	90.23%	93.46%	95.02%	2319.73
8,000	No	85.12%	91.19%	93.33%	864.96

Table 4: Boosting Comparison—Boosted is better

6.5 Validation

When testing neural networks, there is often a splitting of the training set into a regular training set and a validation set. The machine can be "overtrained", meaning that the neural network is being used to detect useless patterns in the training set itself, such as noise, which may actually degrade performance. By training first on the regular training set and monitoring the performance on a separate validation set, one can stop the training at useful times (when the error rate on the validation set increases). Then, the validation data is incorporated and the machine is trained for a few more iterations so as to not waste data. Some experimentation was done and preliminary results suggest little need for a validation set in this case due to paucity of the data per class relative to benchmark datasets such as the MNIST(see Section 4.5).

7 Integration in Practical Recognition System

The author is currently involved in a project to create a commercial handwriting recognition software. This will be assembled by December and likely commercialized by the beginning of next year. The character recognizer portion will be essentially this modified version LeNet 5.

7.1 Conversion to C/C++

The commercial application recently noted necessitated the conversion of LeNet 5 from Lush into C/C++. While Lush is a good language for development, it is not as good as C in a commercial application, since the runtime is necessary but rarely present on user systems. In the interest

[2]50,000 samples compared with 5,676 for the second and 29,238 for the third

of code size and compatibility with existing C/C++ project code, it has been determined that LeNet should be translated into C/C++. The porting process reveals that LeNet is remarkably modularizable.

At present, the translation of the forward propagation portions of the LeNet code entirely to C++ as well as the data source/input modules in C is complete[3]. These modules have been fully tested and also include the improved built-in transforms (see Section 4.5). There are also a number of important improvements in terms of performance gains. The Lush implementation required the entire dataset to be loaded into virtual memory using a matrix, while the C implementation uses pointers to access the elements and hence takes advantage of the buffering of the file system. The file system limitations are usually around 4 gigabytes. Most modern workstations do not possess such amounts of RAM. The modifications thus provide improved flexibility, allowing for larger datasets at a sacrifice of 1-2% speed. For example, with 2,000 test samples, the Lush implementation took 171 seconds for an iteration while our new implementation took 174 seconds. The testing of a single character including creation and destruction of the machine takes .130 seconds. When processing real input, the machine's creation and destruction would not be a factor and hence would perform even faster.

The completion of the forward propagation system in C++ means it is capable of testing data using pre-trained machines. A file saves the state of the machine and makes it possible to train the machine in the Lush system, and then test the data by loading the saved file into the C system. Since the final version is not likely to require training by the end user, the project is currently ready for integration, pending final testing.

7.2 Integration into the Recognizer System

The online recognizer system that is being developed is comprised of three main portions. One is the shape recognizer, which serves as the topmost layer and predicts the identity of an input word and where the segmentation should occur within it. The next layer is the segmentation/bigram layer in which the characters are segmented and sent to the character recognizer. The character recognizer is the system that is described in this paper. It accepts an input comprised of a coordinate

[3]C++ was used for the bulk of the program as the original Lush code was heavily object oriented. C was used for the data source code as that part is also used heavily in the training which includes back propagation portions which is still in Lush and Lush can only handle inline C code

set, plotting this input and feeding it into the pre-trained machine for testing. The outputs are associated with confidence values. The information is then fed back up the tree to the bigram system, a statistical system to test for the likelihood of two characters being adjacent in actual text as well as their relative orientation. Several passes can quickly isolate the correct answer as the decision trees are traversed.

8 Future Work

8.1 Convolutional Kernels

The first layer of convolutions for LeNet consists of six feature units meant to extract certain vital features. The use of convolutional kernels to detect features such as edges or corners has been studied extensively in the image processing and computer vision fields. Experimentation was done with 3×3 kernels in an effort to see if hard coding the kernels might result in improved accuracy. Though there were slight initial gains in accuracy, later iterations showed the current system performing slightly better. These are as yet inconclusive results. Examination of other areas may provide better optimization. There are plans for further experimentation with hard coded 5×5 and 7×7 kernels in the future.

8.2 Integration of Other Networks

Online recognition should be able to provide other handwriting-relevant information such as stroke order, stroke velocity, etc. This suggests testing TDNNs and other networks which may in fact provide better results than convolutional networks in online recognition. Then, the networks can be integrated through heuristics designed to effectively decide when to use any given network to provide the best possible result.

8.3 Loss Function

The loss function that is used in LeNet 5 is the Mean Squared Error (MSE). Studies by Simard and others demonstrate that the Cross Entropy (CE) function may be superior [1]. On the MNIST database, Simard was able to attain 0.4% error compared to LeCun's 0.7% error. There is reason to believe that the loss function is responsible for most of the differential. Exploration in this area is proceeding although results will not be available until the CE function is fully implemented and

tested.

In addition, frequently confused classes can be examined and the difference between their gradients obtained. The difference between the gradients would be due to such things as the way that a hook or curve is drawn, as illustrated by the often confused 'g' and 'q'. The negative of the difference can be added as a term to the loss function so that when the loss function is minimized, the two classes will be pushed apart. Emphasis on local feature learning can be used in this way to separate difficult-to-distinguish classes. This should provide some improvement, although it does violate the ideal of *minimal human intervention*. Implementation of this idea will occur in the future once more automated methods have been exhausted.

9 Conclusion

With these modifications, LeNet 5 and other convolutional networks are capable of discerning far larger character sets than before. Changes such as in-machine transforms, elastic distortions, boosting, more feature units, classifiers, etc., furnished significant gains in accuracy, training efficiency, or both. Difficulties included negative or inconclusive results with hard coded convolutional kernels and increases in the number of input planes.

The final system[4] attained 93.21% accuracy for the top class, 95.46% for top 3, and 97.13% for top 5, compared with the initial accuracy of 55%. This is the best known result to date upon the Unipen dataset[5]. It is also one of the first systems of its kind, demonstrating that a practical and robust character recognition system for a wide character set can be built from a convolutional neural network.

[4]The final system employed the wedist classer, used 7 affine templates per sample, 3 blur/elastic distortions per sample, 63 feature units in second layer, with 4 transforms on each testing output, boosted with 3 copies and 50,000 training samples with 8,000 testing samples in 100 iterations

[5]Another study [10] was able to attain recognition rates of 85.6% and 84.4% respectively for lower case and lower-case extracted from phrases. These are than what this study attains and also deals with a smaller character set, namely the lower case alphabetical characters rather than the entire printbale ASCII set. They used a fuzzy representation technique in combination with a multilayered, backpropagating neural network.

References

[1] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis," In *Proc. 7th International Conference on Document Analysis and Recognition vol. 2*, 2003.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE, vol. 86, no. 5*, pp. 755-824, May 1998.

[3] Y. LeCun, "The MNIST database of handwritten digits" [Online database], Jun 1974, [2004 May 29], Available HTTP: http://yann.lecun.com/exdb/mnist/.

[4] L. R. B. Schomaker and L. Vuurpijl, "The UNIPEN Project" [Online database], Sep 2001, [2004 Jun 15], Available HTTP: http://hwr.nici.kun.nl/unipen.

[5] G. Rätsch, B. Schölkopf, S. Mika, and K.-R. Müller. "SVM and Boosting: One class," *Technical Report 119, GMD FIRST*. Berlin, November 2000

[6] B. Widrow and M. E. Hoff, "Adaptive Switching Circuits," In *Convention Record: Western Electronic Show and Convention, part 4*, 1960, pp. 96-104.

[7] B. Widrow, R. G. Winter, and R. A.Baxter, "Learning phenomena in layered neural networks," In *Proc. 1st Int. Conference Neural Nets, vol. 2*, 1987, pp. 411.

[8] B. Widrow and S. Stearns, *Adaptive Signal Processing*. New York, NY: Prentice Hall, 1985.

[9] M. Banko and E. Brill, "Mitigating the Paucity-of-Date Problem: Exploring the Effect of Training Corpus Size on Classifier Performance for Natural Language Processing," In *Proc. Conf. Human Language Technology*, 2001.

[10] M. Parizeau, A. Lemieux, and C. Gagné, "Character Recognition Experiments using Unipen Data," In *Proc. ICDAR 2001*, 2001.

A Feature Table

class	label	descending	ascending	loop up-hlf	loop lw-hlf	double loop
0	A	0	0	1	-1	-1
1	B	0	0	1	1	1
2	C	0	0	0	-1	0
3	D	-1	-1	0	0	0
4	E	0	0	-1	-1	0
5	F	1	0	-1	-1	0
6	G	0	0	0	-1	-1
7	H	0	0	-1	0	-1
8	I	1	1	-1	-1	0

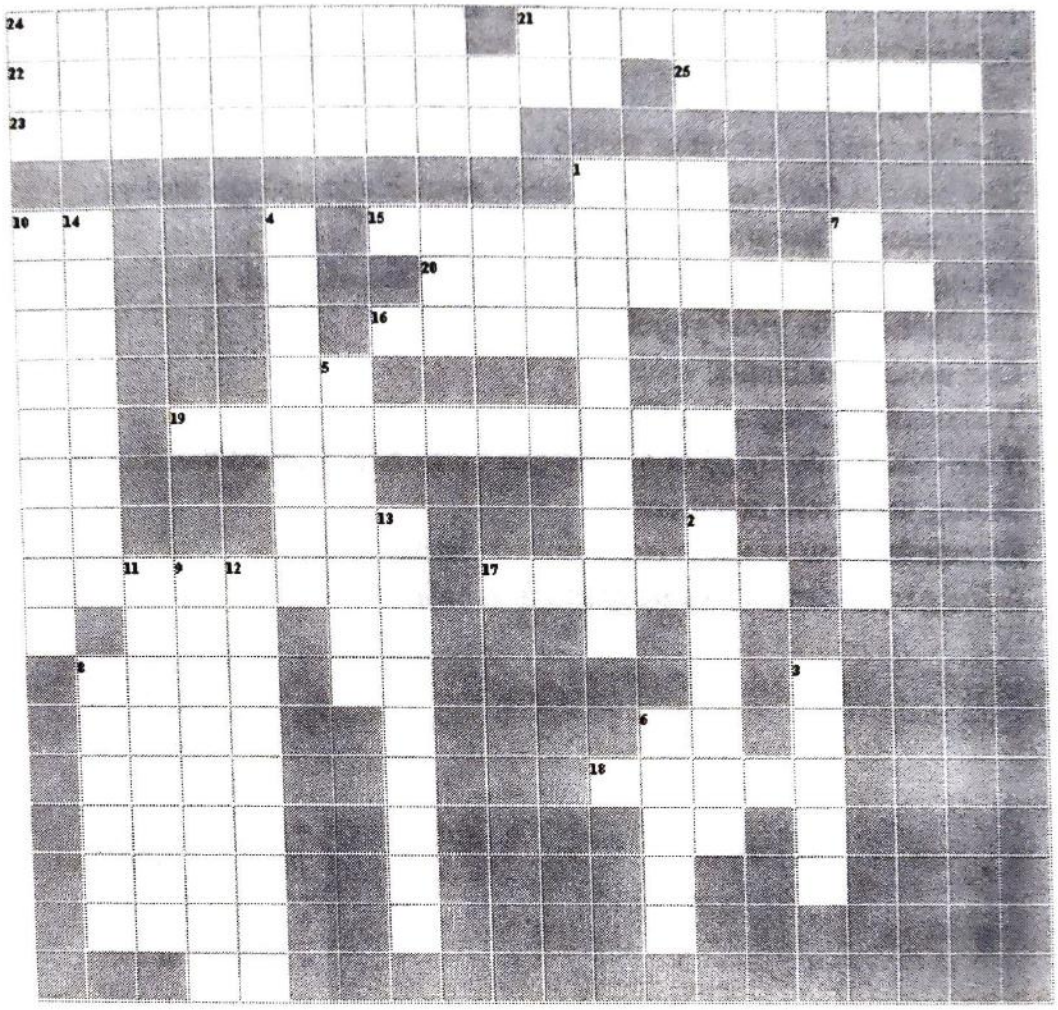
B Sample Lush Code

This is a loop for training the network in the main Lush script file.

```
;; this goes at about 25 examples per second on a PIIIM 800MHz
(de doit (n)
  (repeat n
    (==> thetrainer train trainingdb trainmeter 0.0001 0)
    (printf "training: ") (flush)
    (==> thetrainer test trainingdb trainmeter)
    (==> trainmeter display)
    (printf " testing: ") (flush)
    (==> thetrainer test testingdb testmeter)
    (==> testmeter display)
    ()))

(print (time (doit 5)))
```

Math Crossword



Down

1. Type of integral that requires constant of integration.
2. Yet another method of finding volume.
3. Another method of finding volume.
4. Type of integral that has infinite bound(s), but a finite area.
5. One of the Greek letters typically used in the precise definition of a limit.
6. At age 10, he was able to baffle his teacher by summing 1 through 100.
7. The mathematical study of the properties that are preserved through deformations, twistings, and stretchings of objects.
8. He has a little theorem and a last theorem.
9. You use his rule of signs when you try to determine the roots of a polynomial equation.
10. Another intrinsic property of vectors.
11. An entire class of primes is named after him.
12. Type of series .
13. An intrinsic property of vectors.
14. Polar curve notorious for looking like a heart.

Across

1. The hardest mathematics competition available to qualifying high school students.
15. He's responsible for the notation dy/dx, d/dx, etc.
16. His theorem is a subset of the Mean-Value Theorem.
17. The hardest mathematics competition available to college students.
18. Technique of Integration (1) .
19. An equation in which only integer solutions are allowed.
20. His fractal is the most popular.
21. One method of finding volume.
22. Technique of Integration (2)
23. If the coordinates (x,y) of a point can be given by a third variable, we are dealing with these equations.
24. His rule lets you evaluate indeterminate limits.
25. A line perpendicular to a tangent line is called...